PHILIPS

P 2093

8088 COPOWER BOARD

REFERENCE MANUAL

5103 993 30522

PREFACE

With the aid of the 8088 Copower Board and its associated software, your P2000C is converted to a mixed 8- and 16-bit microcomputer, which is capable of using the MS-DOS operating system, running on an 8088 processor, and the CP/M-80 operating system, running on the original Z80 processor.

This manual has been structured so that you will be led from the start, through the preliminaries - the contents of the package, configuration of the system and its operation - on to the more technical aspects of MS-DOS and CP/M, and their utility programs. The manual finishes with detailed hardware information and service requirements.

If you are a newcomer to microcomputers, you will find the first three chapters of most interest. The more technical features of the operating systems will mainly concern the more experienced user.

The following documents also contain useful information:

Microsoft MS-DOS User Guide
P2000C System Reference and Service Manual
P2519 CP/M Reference Manual

ACKNOWLEDGEMENTS

IBM is a trademark of International Business Machines.

Z80 is a trademark of Zilog, Inc.

MS is a registered trademark of Microsoft Corporation.

CP/M is a registered trademark of Digital Research Inc.

WordStar is a trademark of MicroPro International Corporation.

TABLE OF CONTENTS

Contents

Contents

Contents

1 INTRODUCTION

1.1 Contents of the Package

If the 8088 Copower board is not already installed in your computer, you will find the following in your package:

- 8088 Copower PCB in one of three sizes: 128K, $\boxed{256K}$ or 512K RAM.

ISTAL →

- Terminal ROM BIOS chip (which gives a 25-line display on your screen).

- CP/M-80 diskette (640K), containing system files and utility programs, but no CP/M system; i.e. it is not a bootable disk.

- MS-DOS diskette (800K), containing system files and standard utilities.

- Microsoft MS-DOS User Guide

- P2093 8088 Copower Board Reference Manual

2

Introduction

1.2 Purpose of the Copower Board

With the addition of the 8088 Copower board and its associated software, the P2000C microcomputer is converted to a mixed 8- and 16-bit computer. The enhanced machine is capable of running the MS-DOS operating system on the 8088 processor or the CP/M-80 operating system on the original Z80 processor.

The Copower board contains memory that can be used as main memory by MS-DOS or, when the computer is running under CP/M, as a RAM disk (instead of the normal RAM disk extension card).

Within the limitations of the P2000C's architecture, the MS-DOS implementation has been made as compatible as possible with the [redacted]-[redacted].

1.3 Introduction to MS-DOS on the P2000C

For the P2000C, version 2.11 of MS-DOS is available. This operating system will, within limits, support IBM-compatible applications.

In this implementation, there are two modes of operation: the P2000C mode, for 7-bit MS-DOS applications, and the PC mode, for 8-bit PC DOS applications. Where necessary, sections of the manual have been split to reflect properly the different features of the two modes.

There are three categories of software to be considered:

- that which is "well behaved" MS-DOS software: using only official MS-DOS functions and requiring no access to the underlying machine. This should run on all machines supporting MS-DOS as long as enough memory and disk storage is provided. This will run on the Copower board in either the P2000C mode or the PC mode. The software must be available on a format of disk supported by the Copower board; i.e. Philips P2000C MS-DOS format or the various IBM PC formats (but not IBM PC AT formats).

- that which is "poorly behaved": using the MS-DOS functions and the authorized features of the IBM PC ROM BIOS. This should run on the Copower board in the PC mode. The software must be available on a suitably formatted disk.

Introduction

- that which is "badly behaved": using unauthorized features of the IBM ROM BIOS or going directly to the hardware of the machine. This includes those programs that directly access video memory. These programs will not run on the P2000C Copower board.

Note: ROM = read only memory; i.e. memory which is permanently in the machine.

Note: BIOS = Basic Input Output System, a piece of software provided by the manufacturer to control the hardware. This is used to insulate the operating system from the fine detail of the underlying hardware. "Misbehaved" software will access this level directly to achieve a higher performance than can be obtained when using the operating system functions.

The P2000C MS-DOS [redacted] disks which have a standard 160K, 180K, 320K or 360K IBM PC format, and write to them, with the restriction that they may not be readable on a standard IBM disk drive. This is because the higher capacity P2000C disk drives have a narrower head width and so the data written to the disk by the P2000C may be too narrow to be read correctly on the IBM PC. The ability to read correctly will be determined to a large extent by the correctness of the mechanical adjustments in the disk drives. (The drives can only be realigned by a suitably skilled service technician.)

1.4 Description of the Board

The 8088 Copower board is an extension card which uses the internal extension slot in the P2000C. The board contains an 8088 processor, together with the expansion capability of an 8087 mathematics processor, and memory available in 128K, 256K and 512K options.

The 8087 mathematics processor is not necessary for the operation of MS-DOS or most application software. Indeed, software that was not written specifically to use the 8087 instructions will not use the 8087 even if it is installed.

However, applications that have been written to take advantage of the 8087 will perform mathematical functions and calculations faster than they would without the coprocessor being present. The 8087's extra instructions (which are not available in the 8088's instruction set) allow faster execution of multiply, divide and other mathematical functions.

The 8088 board can hold varying amounts of RAM memory depending on the number and type of RAM memory chips installed. It is capable of supporting 64k-bit or 256k-bit memory chips. When the board is full with 64k-bit chips, it will provide 128K of memory. When it contains 256k-bit chips, it can provide either 256K or 512K of memory.

2 INSTALLATION AND CONFIGURATION

2.1 Hardware Installation

If the 8088 Copower board and terminal software are not already installed in your computer, refer to Chapter 10.

The essential items for installation are a P2000C with 640K disk drives, the 8088 Copower board, and the terminal software (which should be version 1.3 or greater).

If you are in doubt about the presence of the terminal software, you can check by holding the ESC key down and pressing the RESET button. If the software is present, you will see on your screen a message showing the terminal software version number. (The system will then go into a terminal test routine, so you should press the RESET button to carry on.)

You can check to see that the Copower board is installed by running the utility program MSBOOT. If the board is not present, you will see the message 'Hardware error, NO ANSWER from extension card'. If your disk is not yet installed (refer to sections 6 and 7 of this chapter for the installion procedure), you will see the message 'File not found: MSKEY.TAB'.

2

Installation and Configuration

2.2 Preparing Disks

Before you go any further, put a write-protect tab on each of your two delivered disks.

2.2.1 Preparing the Copower Disks

The first thing to do is to make back-up copies of your original two disks supplied with the Copower board (the CP/M disk and the MS-DOS disk), using MSUTIL as described in Chapter 2.4.2.

2.2.2 Preparing a CP/M Work Copy

Make a copy of your normal CP/M system disk which should contain the following files:

cpm61.com
cpm62.com
cpm63.com
config.com
config.msg
config.dat
pip.com
stat.com
submit.com
util.com
util.msg

The disk should have at least [illegible] of free space. You can use SYSGEN to put the bootstrap information on the disk.

Installation and Configuration

2.3 Disk Contents

2.3.1 CP/M Copower Disk

TEST SOFTWARE

test88.com - a Copower board test program

RAM DISK SUPPORT

cbios61.com - allow use of Copower board
cbios62.com as RAM DISK from CP/M
cbios63.com
instcpm.sub - submit file to install

MS-DOS SUPPORT

msconf.com - a modified version of CONFIG
msconf.dat - data files for MSCONF

msboot.com - a secondary bootstrap program to load MS-DOS into the Copower board and transfer control to MS-DOS

ibmkey.tab - special key translate table for PC mode

phikey.tab - special key translate table for P2000C mode

instibm.sub - submit file to install in PC mode instphi.sub - submit file to install in P2000C mode

MSUTIL AND ASSOCIATED FILES

- msutil.com - special format and copy program, and hard disk installation
- msutil.msg - message file for MSUTIL
- msutilt.mac - source code for disk tables
- msutilm.mac - source code for messages
  - these files can be altered to add new formats of diskette, or translate messages

MSUTILM and MSUTILT are written in Z80 mnemonics. They are loaded and linked, with the Microsoft utilities M80 and L80, in this way:

```
M80 =MSUTILM
L80 /P:100,MSUTILM,MSUTIL.MSG/N/E
```

2.3.2 MS-DOS Copower Disk

STANDARD MS-DOS FILES

- msdos.sys (hidden file)
- io.sys (hidden file)
- command.com
- edlin.com
- debug.com
- link.exe
- chkdsk.com
- sys.com
- diskcopy.com
- recover.com
- print.com
- more.com
- find.exe
- exe2bin.exe
- fc.exe

P2000C-SPECIFIC FILES

format.com - modified for P2000C format

config.sys - adjusted to suit MS-DOS on P2000C

ansi.sys - an ANSI terminal output driver (no keyboard reassignment)

clr.com - clear screen in P2000C mode

reboot.com - restart MS-DOS without return to CP/M

mssort.exe - a sort program with straight collating sequence, can be renamed sort.exe

pcsort.exe - a sort program with the IBM PC collating sequence, can be renamed sort.exe

instibm.bat - a command file to set config.sys for PC mode

instibm.sys - a PC mode copy of config.sys

instphi.bat - a command file to set config.sys for P2000C mode

instphi.sys - a P2000C mode copy of config.sys

2.4 Disk Formatting and Copying

2.4.1 Formatting

FLOPPY DISKS

Before you can use floppy disks under MS-DOS, they must be formatted and have a directory installed, and have some other system information present. (Chapter 9.4 contains more detail.) For new disks, this can be done under the CP/M operating system using the MSUTIL program, or, under MS-DOS, using the FORMAT program.

You can format disks to an IBM PC format by using MSUTIL. (Note that these disks cannot be used as system disks.) To get a higher storage capacity with the P2000C format of 800K, use either MSUTIL or FORMAT.

In answer to the CP/M prompt A>, type MSUTIL. You will then see the MSUTIL main menu. Select option 1, FORMAT FLOPPY DISK. You will see the following message:

```
Enter sequentially the DRIVE NUMBER(S)
containing the disks to be formatted,
terminate with <CR>
exit with <ESC> or <0>
Formatting continues cyclically until an
empty drive is found

Drive(s) to format (1 to 4) =
```

After you have entered the drive number (or numbers), you will see the next prompt:

```
- Select DESTINATION disk type
  1 - P2000C DOUBLE SIDED
  2 - P2000C 160K CP/M
  3 - P2000M CP/M
  4 - P2500 300K CP/M
  5 - P2500 600K CP/M
  6 - P3500 CP/M
  7 - P5020 320K CP/M
  8 - IBM PC
  9 - RAINBOW 400K CP/M
  A - KAYPRO 200K CP/M
```

Now, you have 10 choices. Normally, you will choose the first option, P2000C DOUBLE SIDED. You will then be asked:

```
- Which SYSTEM?
  1 - CP/M
  2 - MS-DOS
```

Choose option 1, CP/M, if you are formatting a CP/M disk, and option 2 if you are formatting an MS-DOS disk.

You will see a warning message before the format program continues. Take care that the correct disk is in the correct drive. The format program will act on any disk, whether that disk is empty or not.

HARD DISKS

A hard disk should be formatted under CP/M in the normal way, i.e. using the UTIL program. Select option 2 from the UTIL main menu, FORMAT HARD DISK.

Note that under MS-DOS it is possible to support larger capacity hard disks than is possible under CP/M-80. So, the hard disk format option of the normal UTIL program can format larger capacities than would normally be used on the P2000C with CP/M.

2.4.2 Copying Floppy Disks

CP/M OR MS-DOS (VARIOUS FORMATS)

With MSUTIL, it is possible to make a copy of a complete disk by choosing the physical copy option; i.e. a copy from track to track. You can also copy a file to a file, if you wish.

If you are running under CP/M, you should use the MSUTIL program to copy disks. This program works even if you do not have a Copower board, and is capable of formatting and copying all the types and formats of disk listed in the selection menus.

If you are running under MS-DOS, you can use the DISKCOPY command. Now, you can copy only MS-DOS disks.

UNDER CP/M

From the main menu of the MSUTIL program select option 2, COPY FLOPPY DISK. Proceed as follows:

- Enter SOURCE drive (1 to 4) : 1
- Enter DESTINATION drive (1 to 4) : 2
- Do you wish to verify while copying (y/n) ? y

(Again, take care that you have the correct disk in the correct drive. It is a good idea to put a write-protect tab on the disk that you want to copy.)

- Select SOURCE disk type:
  1 - P2000C DOUBLE SIDED
  2 - P2000C 160K CP/M
  3 - P2000M CP/M
  4 - P2500 300K CP/M
  5 - P2500 600K CP/M
  6 - P3500 CP/M
  7 - P5020 320K CP/M
  8 - IBM PC
  9 - RAINBOW 400K CP/M
  A - KAYPRO 200K CP/M

Now, you have 10 choices. Normally, you will choose the first option, P2000C DOUBLE SIDED. You will then be asked:

- Which SYSTEM?
  1 - CP/M
  2 - MS-DOS

Choose option 1, CP/M, if you are copying a CP/M disk, and option 2 if you are copying an MS-DOS disk. Now, you will be asked to choose the type of disk that will be the destination. You will see the same 10 options as for the source. Again, you will normally choose option 1, P2000C DOUBLE SIDED.

You will be asked:

- Which SYSTEM?
  1 - CP/M
  2 - MS-DOS

Choose 1 or 2, as applicable. In answer to the next prompt, 'Make PHYSICAL (track to track) copy (y/n)?', answer 'y'. You will finally be warned that your destination disk will be deleted (have you got the right disk in your destination drive?), and told to press any key to copy.

UNDER MS-DOS

To copy a complete P2000C 800K-format disk under MS-DOS, use the DISKCOPY command. To copy files, use the COPY command. Files can be copied from and to either P2000C 800K- or IBM PC-format disks.

You will find all the details in the MS-DOS User Guide.

2.5 Installing as a RAM Disk under CP/M

Although the Copower board provides the 16-bit processing capability for MS-DOS, it has another function. It acts as a RAM disk for the CP/M-80 operating system.

To use the board as a RAM disk, the CP/M-80 system must be configured using new versions of the supplied CBIOS files. These files also support the normal RAM disk extension card.

It should be remembered that the amount of memory on the Copower board may differ from that on the normal RAM disk extension card, and so the supported disk size may be different.

The data content of the Copower board, when used as a RAM disk, is preserved during a reset and restart of the CP/M system.

To install the new CBIOS files on the work copy of your CP/M system disk (as described in Chapter 2.2.2), you should boot up with the work copy in drive A and the back-up copy of your Copower CP/M disk, containing the support files, in drive B, and then use the submit file INSTCPM.SUB to transfer the files from drive B to drive A. The submit file will invoke the CONFIG program to ensure that you reconfigure using the new CBIOS files.

The command to do this is:

SUBMIT B:INSTCPM

12

Installation and Configuration

Using CONFIG, don't forget to define a disk volume for the RAM disk, and also to write the new system information onto a disk that you intend to use later as a normal CP/M system disk.

2.6 Installing MS-DOS (PC Mode)

To run programs written for the IBM PC, you have to use this configuration.

2.6.1 What to do under CP/M

With the CP/M work copy (as described in Chapter 2.2.2) in drive A, the back-up copy of the Copower CP/M disk in drive B, and the system showing the prompt A>, it is only necessary to issue the following command to make the CP/M work disk suitable to load and start MS-DOS:

SUBMIT B:INSTIBM

The submit file will invoke MSCONF, so you should then select the keyboard and printer that you require and complete the configuration so that the new information is written back onto the disk. After a RESET the CP/M system should show on the screen the names of the printer, keyboard and video that you have selected. This can be used as a check that you have really configured as you expected.

The initial configuration will only support two floppy disk drives. But, if you have a hard disk, add it to the disk configuration.

2.6.2 What to do under MS-DOS

Boot CP/M from your work copy and type 'MSBOOT'. Then insert the back-up copy of the MS-DOS Copower disk and type any key. MS-DOS will be booted. Type the date and the time, followed by a carriage return.

Give the command 'INSTIBM', which will copy the contents of the text file INSTIBM.SYS into CONFIG.SYS and copy PCSORT.EXE into SORT.EXE. Then, the REBOOT program will be run to ensure that MSDOS uses the new CONFIG.SYS file.

It is sensible to familiarize yourself with the content of the CONFIG.SYS file to check that the ANSI driver is installed. This can be done with the command:

```
TYPE CONFIG.SYS
```

which will list the file, one line of which should say:

```
DEVICE=ANSI.SYS
```

2.7 Installing MS-DOS (P2000C Mode)

To run (non-IBM) MS-DOS programs, it is advisable to use this installation.

2.7.1 What to do under CP/M

With the CP/M work copy in drive A, the back-up copy of the Copower CP/M disk in drive B, and the system showing the prompt A>, it is only necessary to issue the following command to make the CP/M work disk suitable to load and start MS-DOS:

```
SUBMIT B:INSTPHI
```

The submit file will invoke CONFIG, so you should then select the keyboard and printer that you require and complete the configuration so that the new information is written back onto the disk. After a RESET the CP/M system should show on the screen the names of the printer, keyboard and video that you have selected. This can be used as a check that you have really configured as you expected.

The initial configuration will only support two floppy disk drives.

2.7.2 What to do under MS-DOS

It is necessary to load the back-up copy of the Copower MS-DOS disk when prompted by the MSBOOT program, and then to use CONFIG.SYS without the line:

```
DEVICE=ANSI.SYS
```

Do this by giving the command 'INSTPHI', which will copy the contents of the text file INSTPHI.SYS into CONFIG.SYS and copy MSORT.EXE into SORT.EXE. You should run REBOOT to ensure that MS-DOS will use the new CONFIG.SYS file.

It is sensible to familiarize yourself with the content of the CONFIG.SYS file so that you can check that 'DEVICE = ANSI.SYS' is not present in the file. Use the command:

```
TYPE CONFIG.SYS
```

which will list the file.

2.8 Sharing a Hard Disk

A hard disk can be added to the system. This disk can be divided into two sections: one for CP/M and one for MS-DOS. There must always be a CP/M section defined, and it must be at the beginning of the disk so that the initial CP/M-80 system can be booted from the hard disk. The CP/M section can be up to 5MB.

The CP/M system disk, from which the system was booted before the MSBOOT program was executed, must be configured so that it contains only one logical disk assigned to the hard disk. The logical disk should be selected as the lower section (either as 2MB low or 5MB low).

After the CP/M hard disk configuration has been defined in the configuration program (either MSCONF or CONFIG, as appropriate), you must carry out a further configuration from within the MSUTIL program to define the starting point of the MS-DOS volume. It will be defined to start at some point within the CP/M logical disk and extend physically upwards to the end of the disk.

It is possible that the CP/M-80 area will finish before the logical end of the hard disk volume. The section of the hard disk which is defined as common to both CP/M and MS-DOS will be made into a file and hidden, so that the CP/M system will not access that section, so allowing the MSBOOT program to determine where the MS-DOS section begins. In this way, MS-DOS can find and load its system files.

Using MSUTIL, it is not possible to define an MS-DOS section which will accidentally overwrite CP/M data that exists within the logical volume of the current configuration. But you should remember that MS-DOS can use any hard disk space (up to 32MB) which is above the configured CP/M logical volume.

2.9 Installing on Hard Disk

2.9.1 CP/M Files

The files from the Copower CP/M disk can be installed onto the hard disk if the normal CP/M system is started from the hard disk as drive A, and the Copower disk is placed in the floppy drive B. It is then only necessary to run the appropriate install submit file.

i.e. SUBMIT B:INSTCPM
or SUBMIT B:INSTPHI
or SUBMIT B:INSTIBM.

You may find it more convenient to perform the commands contained in the submit files manually, particularly if you wish to install onto the hard disk when drive B is not configured for a floppy disk.

Before installing MS-DOS on the hard disk, you must use MSUTIL to define the portion of the hard disk that is to be used by MS-DOS.

2.9.2 MS-DOS Files

To install the MS-DOS files on the hard disk, you should start MS-DOS from a floppy disk using MSBOOT. The hard disk will be recognized as drive C if it has already been partitioned by MSUTIL. The hard disk must be switched on before the MSBOOT program is used.

The hidden MS-DOS system files are then transferred to the hard disk with the MS-DOS command 'SYS'. Make sure that the current disk is A; i.e. showing 'A>' as the prompt. Then enter the command SYS C: which will copy the system files to the hard disk.

Next, the program COMMAND.COM must be copied to the hard disk with the following command:

COPY COMMAND.COM C:

The other files should then be copied with the command:

COPY *.* C:

Note: The SYS command must be used before any files are copied to the hard disk because the hidden system files MUST be at the beginning. If this is not done, MS-DOS will not be able to find the files and will give an error message.

2.10 APPLICATION SOFTWARE

Many application packages use copy protection schemes to ensure that a user is unable to copy the disks, and so reduce the incidence of "software piracy". Some of the protection schemes may make direct access to features of one manufacturer's hardware, and therefore that software may not operate on the P2000C. (Access directly to the disk controller from the 8088 software is sometimes used, which under the P2000C will not work because the controller can only be accessed from the Z80 processor.)

A popular copy protection scheme uses specially formatted disks, and the application checks that the format is of this special type. A user cannot format a disk to this format. So, a copy would lose its special format and this absence would be detected by the application as it ran.

The two different modes of operating under MS-DOS on the P2000C have to be considered when installing application software.

P2000C MODE

Some software has message files that are coded in 7-bit ISO characters with national variations; for example, an "A umlaut" in German software would be coded with the same value as a square bracket under ASCII. This software would have to be used under the P2000C mode.

The software may need to be configured so that it recognizes the control codes generated by the function keys - the cursor keys, for example. These codes would be similar to those recognized by "Wordstar"; that is, control characters such as CTRL-X.

The software may need to be configured so that it generates the correct screen control characters and escape sequences. In this mode they will be the normal P2000C codes as described in the P2000C System Reference Manual. The software will not work correctly in this mode if it generates ANSI escape sequences.

PC MODE

Some software has message files that are coded in the 8-bit IBM PC character set with national characters; for example, an "A umlaut" in German software, represented by an entry in the top half of the code set above the ASCII characters. This software would have to be used under the PC mode.

The software may need to be configured so that it recognizes the control codes generated by function keys - the cursor keys, for example. These codes will be similar to those generated by the IBM PC ROM BIOS.

The software may need to be configured so that it generates the ANSI screen control escape sequences or BIOS calls. If the software generates ANSI escape sequences then the MS-DOS system must have ANSI.SYS selected. (This is done by having the line

DEVICE=ANSI.SYS

in the file "CONFIG.SYS".)

If the software generates BIOS calls to perform screen handling then it may not be necessary to have ANSI.SYS installed.

The ANSI.SYS code on the P2000C has a second function. This is to filter control characters and cause them to be displayed as they would on an IBM PC, instead of allowing them through to control the P2000C terminal. This second function means that although ANSI escape sequences are not required, it may be necessary to use ANSI.SYS.

Software that does not generate control sequences or ROM BIOS calls, but instead uses the video memory directly, will not operate.

3 USING MS-DOS

Within the restrictions of the architecture of the P2000C, the MS-DOS implementation has been made as compatible as possible with that of the IBM PC. The Microsoft MS-DOS User Guide contains a full description of the operating system and how it should be used.

3.1 Disk Drive Naming Conventions

MS-DOS can access two internal 96 tpi double sided floppy disks (as supplied on the P2000C model with 640K disk drives) and an attached hard disk. The system cannot be configured for external floppy disk drives or for internal 48 tpi single sided floppy disk drives.

There are two predefined configurations. The process of configuration is automatic, depending on whether the system is booted from a floppy disk or from a hard disk.

Boot Device	Disk Identifiers		
	A	B	C
floppy disk	floppy 1	floppy 2	hard disk
hard disk	hard disk	floppy 1	floppy 2

3.2 Booting, Rebooting, Reconfiguring

Before we talk about the bootstrap procedures, there are two points to remember. Firstly, the system boots by default from floppy disk; and secondly, if you have a hard disk, it must be switched on before you run MSBOOT.

3.2.1 Booting from Floppy Disk

To boot MS-DOS from a floppy disk, do the following:

- put the MS-DOS work copy in drive 1 and leave the door open

- put the CP/M work copy in drive 2

- press the RESET button

- in response to the CP/M prompt A>, type msboot. You will see the message:

Insert MS-DOS disk in drive 1, type return to boot (or exit by CTRL-C)

- You already have the MS-DOS disk in drive 1, so close the door and press the carriage return key

Now, the MS-DOS system will be loaded and you will see a request to enter the date. From this point, you can refer to the MS-DOS User Guide for details.

3.2.2 Booting from Hard Disk

After installing the CP/M and MS-DOS files, as described in Chapter 2.9, it is only necessary to boot CP/M from the hard disk and run the MSBOOT program, which will start MS-DOS. The MSBOOT program can be started automatically with the CP/M autostart feature, so that it is only necessary to reset the P2000C - CP/M and MS-DOS will be restarted automatically. (To set the autostart feature, use CONFIG when in P2000C mode and MSCONF in PC mode.)

You must give either the INSTIBM or INSTPHI command to make sure that the MS-DOS, which is booted from the hard disk, is configured correctly. This need only be done on the first occasion you boot.

3.2.3 Rebooting MS-DOS

To reboot the system, you can either press the RESET button and restart in the normal way or, from MS-DOS, you can run the REBOOT program, which will restart MS-DOS ~~without the need to go~~ back first to CP/M.

3.2.4 Reconfiguring the System for Keyboard, Printer and Screen

The important point about reconfiguration is that the correct program should be used depending on the mode you are in. If you are in P2000C mode, you should run CONFIG. If you are in PC mode, you should run MSCONF.

If you are reconfiguring MS-DOS, it may be necessary to run INSTIBM or INSTPHI, so that CONFIG.SYS may be set correctly for the appropriate mode.

3.3 Backup/Restore

For data files that do not exceed the storage capacity of one floppy disk (approximately 800K), you can use the normal MS-DOS COPY command to transfer data from a hard disk to a floppy disk, or vice versa.

If this is not adequate, you can get a special backup/restore package from your Philips P2000C dealer. This package is specifically designed to copy and restore large files.

3.4 The Keyboard

3.4.1 P2000C Mode

In this mode, keys are used in exactly the same way as under the P2000C CP/M system. No special function keys are defined other than the simple 'Wordstar keys' defined by the standard keyboard configuration of the CP/M-80 configuration program. These keys have normal control functions; for example, CTRL-C.

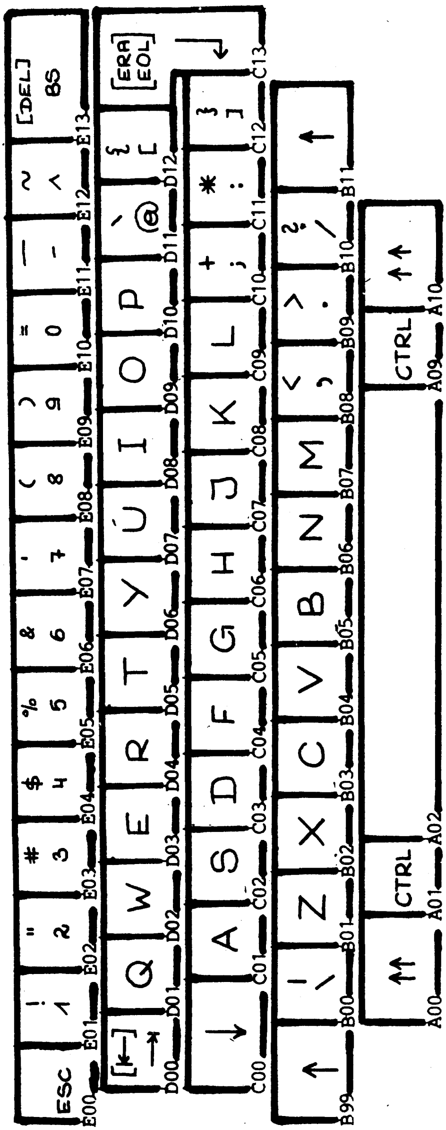
The MS-DOS special editing functions are not available when you are running in P2000C mode. These keys generate 7-bit ISO codes with national variations. If you want to configure some additional 8-bit codes on the supershift level, you must use the keyboard table editor in the CONFIG program. Some useful keys are listed in Chapter 6.9.

3.4.2 IBM PC Mode

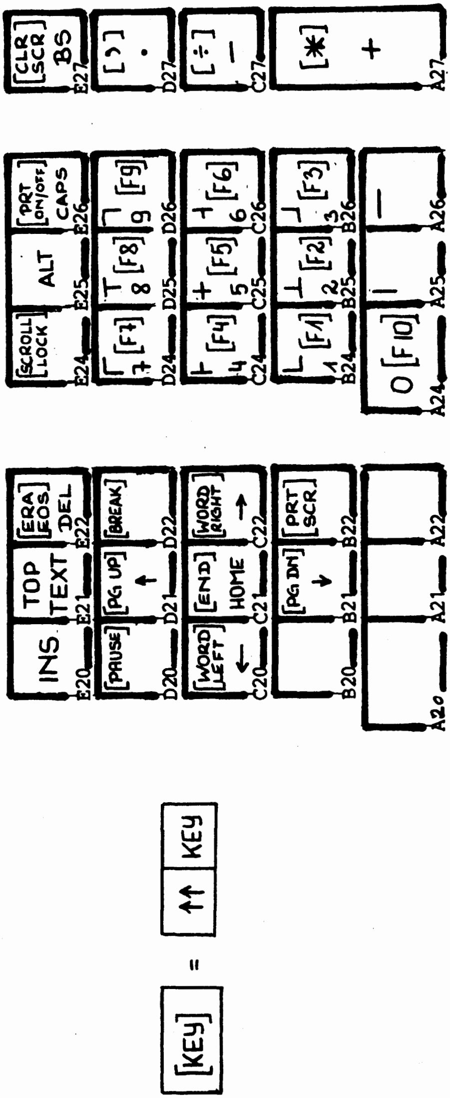
In this mode, keys generate the 8-bit codes which are defined for the IBM PC. Function keys are generated in a similar way as on the IBM PC, and will be recognized as such by MS-DOS. These keys will perform the standard MS-DOS special editing functions as described in the MS-DOS User Guide.

Keyboard national versions, equivalent to those of the IBM PC, are made by reconfiguring the underlying CP/M-80 system using MSCONF.

Using MS-DOS


ESC
[DEL] BS
[ERA EOL]
CTRL
CTRL
E00
E01
E02
E03
E04
E05
E06
E07
E08
E09
E10
E11
E12
E13
D00
D01
D02
D03
D04
D05
D06
D07
D08
D09
D10
D11
D12
C00
C01
C02
C03
C04
C05
C06
C07
C08
C09
C10
C11
C12
C13
B99
B00
B01
B02
B03
B04
B05
B06
B07
B08
B09
B10
B11
A00
A01
A02
A09
A10

Using MS-DOS


INS
TOP TEXT
[ERA EOS] DEL
[PAUSE]
[PG UP]
[BREAK]
[WORD LEFT]
[END] HOME
[WORD RIGHT]
[PG DN]
[PRT SCR]
E20
E21
E22
D20
D21
D22
C20
C21
C22
B20
B21
B22
A20
A21
A22
[SCROLL LOCK]
ALT
[PRT ON/OFF] CAPS
7 [F7]
8 [F8]
9 [F9]
4 [F4]
5 [F5]
6 [F6]
1 [F1]
2 [F2]
3 [F3]
0 [F10]
E24
E25
E26
D24
D25
D26
C24
C25
C26
B24
B25
B26
A24
A25
A26
[CLR SCR] BS
[,] .
[÷] −
[*] +
E27
D27
C27
A27
[KEY] = ↑↑ KEY

FUNCTION KEYS

These are available in PC mode by entering one of the numbers on the numeric pad together with the supershift key [^^]. For example, F10 is [^^] with [0] and F1 is [^^] with [1].

Special functions are also available by entering the appropriate key, usually together with supershift, as you can see in the diagram. In the list below, we indicate those keys that should be pressed in conjunction with the supershift key.

```
     B21 Cursor down
[^^] B21 Page down
[^^] B22 Print screen
[^^] C13 Erase to end of line
     C20 Cursor left
[^^] C20 Word left
[^^] C21 End
     C21 Home
     C22 Cursor right
[^^] C22 Word right
[^^] D20 Pause
     D21 Cursor up
[^^] D21 Page up
[^^] D22 Break
```

E20 Insert (with or without supershift)
E21 Top of text and home (with or without supershift)
E22 Delete
[^^] E22 Erase to end of screen
[^^] E24 Scroll lock
[^^] E26 Printer on/off
E26 Capslock
[^^] E27 Clear screen

ALT KEYS

The ALT keys are available for the alphanumeric and numeric keys.

[^^] A...Z is equivalent to ALT A...Z

Note: When running with a national version, the letter may be marked on a different key position to that of the IBM PC US version. Use the letter as marked on the keyboard, not the key with the same position as that of the IBM version.

[^^] 1...0,-,^ is eqivalent to ALT 1...0,-,=

Note: The [-] and [^] symbols are marked on the P2000C UK keyboard. On other national versions, use the two keys that are in these positions. These keys are the two between the [0] and [backspace] keys on the top line of the keyboard.

[ALT]
[n]...[n] (up to 3 digits from the numeric pad)
[non-numeric, e.g. CR]

is equivalent to: hold ALT
enter number from numeric pad
release ALT

and will generate one character with ASCII value nnn and position code 0.

CTRL KEYS

The use of the CTRL key together with a number will not generate the same code as the IBM PC because of restrictions of the underlying P2000C terminal software and keyboard handling. The generated codes will conflict with CTRL and [letter].

The P2000C restrictions give the following:

[ESC] = [CTRL] ['['] = [CTRL][;]
[CR] = [CTRL] [m] = [CTRL] [-]
[BS] = [CTRL] [h]
[TAB] = [CTRL] [i]

ENTERING CHARACTERS NOT MARKED ON THE KEYBOARD

It is possible to enter any ASCII value from 0 to 255 with the [ALT] key, followed by up to 3 digits and ended with a non-numeric character.

You can also enter some characters at the supershift level, or shift/supershift level, that are not available on the normal national version keyboard. On any national version keyboard, those keys which are not alphabetic and are on the main area (excluding the top row) will at the supershift, or shift/supershift level, generate the symbols defined in the same position on the UK keyboard at the unshifted or normal shift level.

For example, the key (P2000C position B00) will generate on the UK keyboard a backslash or vertical bar at the normal levels. National versions will not, but will at the supershift or shift/supershift levels. These keys are at the following positions:

B00, B08, B09, B10
C10, C11, C12
D11, D12

unless that position generates a letter [A...Z] on the national version keyboard.

3.5 CHARACTER SETS

3.5.1 P2000C Mode

The character set for the MS-DOS P2000C mode is the 7-bit ISO code with national variations as described in the P2000C CP/M Reference Manual.

3.5.2 PC Mode

The character set for the MS-DOS PC mode is the 8-bit IBM PC character set, but with the restriction that not all characters can be represented on the screen due to the restrictions imposed by the inbuilt character generator of the P2000C. The nearest equivalent character will be used to represent the required character; for example, the Peseta character will be represented by a capital "P" on the screen but will print according to the level of support provided by the printer. Most printers do not fully support the IBM PC character set.

The user can redefine the displayed or printed characters by editing the applicable tables in the "MSCONF.DAT" file, if desired, by using the table editing facility of "MSCONF.COM". (It uses the same interface as the CP/M CONFIG program.)

Characters with an internal representation in the control character range (values from 0 to 32) cannot be displayed unless they pass through the ANSI driver, which will switch into mosaic graphic mode and display the characters after adding an offset of 32.

Programs that go directly to the BIOS for output of characters must perform this conversion through 'mosaic mode' themselves. (See the P2000C System Reference Manual for more information on mosaic graphics.)

3.6 Clear Screen Command

3.6.1 CLS (in PC Mode)

The inbuilt command CLS of COMMAND.COM is used in PC mode to clear the screen. This is documented fully in the associated Microsoft documentation. In the P2000C MS-DOS operating system this command is configured to generate the ANSI escape sequence for clear screen.

Use of CLR in PC mode will not clear the screen but will display the characters "2J" on the screen.

3.6.2 CLR (in P2000C Mode)

The program CLR.COM is used instead of CLS to clear the screen when operating in P2000C mode. This is because the operating system command program "COMMAND.COM" can only contain one version of the CLS command, which is configured to operate in the PC mode.

Use of CLS in P2000C mode will not clear the screen but will display a character on the screen which looks like the cursor.

3.7 System Parameters (CONFIG.SYS)

The text file CONFIG.SYS contains a number of commands which are used during the MS-DOS startup sequence to define some aspects of how the system will be configured. For further details of this file, and the available options, consult the MS-DOS User Guide.

The file is supplied with the following:

device=ansi.sys	- this causes the system to load the ANSI screen output driver on startup.
buffers=10	- this defines the number of file buffers that are to be used.
files=10	- this defines the maximum number of files that can be used at any one time. This setting is adequate for most usage but can be increased as needed.
country=49	- this is set for Germany. (For the UK, it is set to 44, for France to 33, etc.) Its main use is to force the system to adopt the European date format.

If you are not sure how to use the system editor EDLIN to modify CONFIG.SYS, the two files INSTIBM.SYS and INSTPHI.SYS (two versions of CONFIG.SYS) are provided to make it easy for you to select an appropriate CONFIG.SYS.

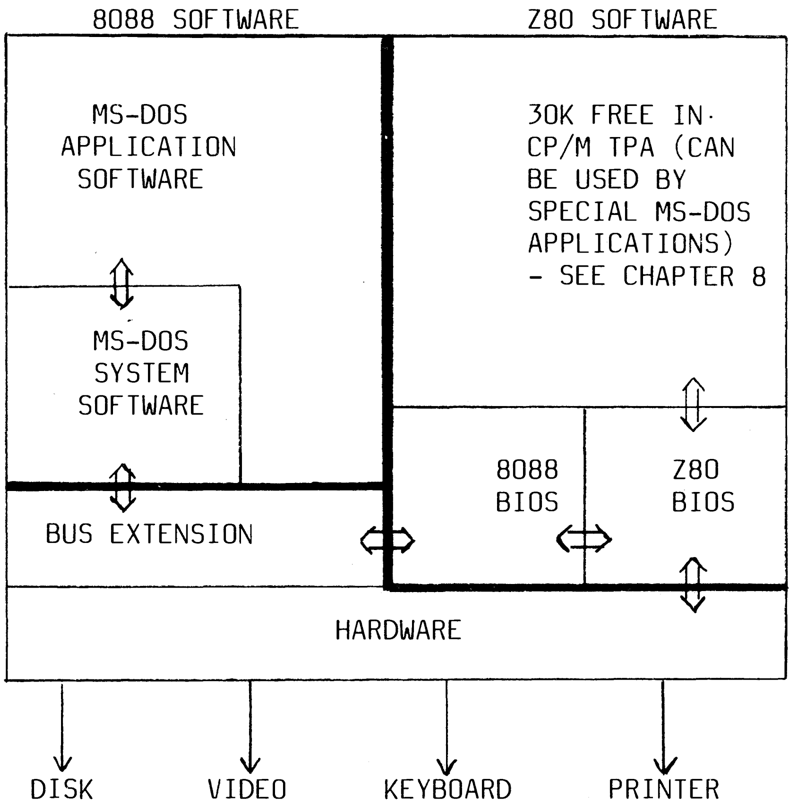
4 MS-DOS ON THE P2000C

4.1 Implementation of MS-DOS

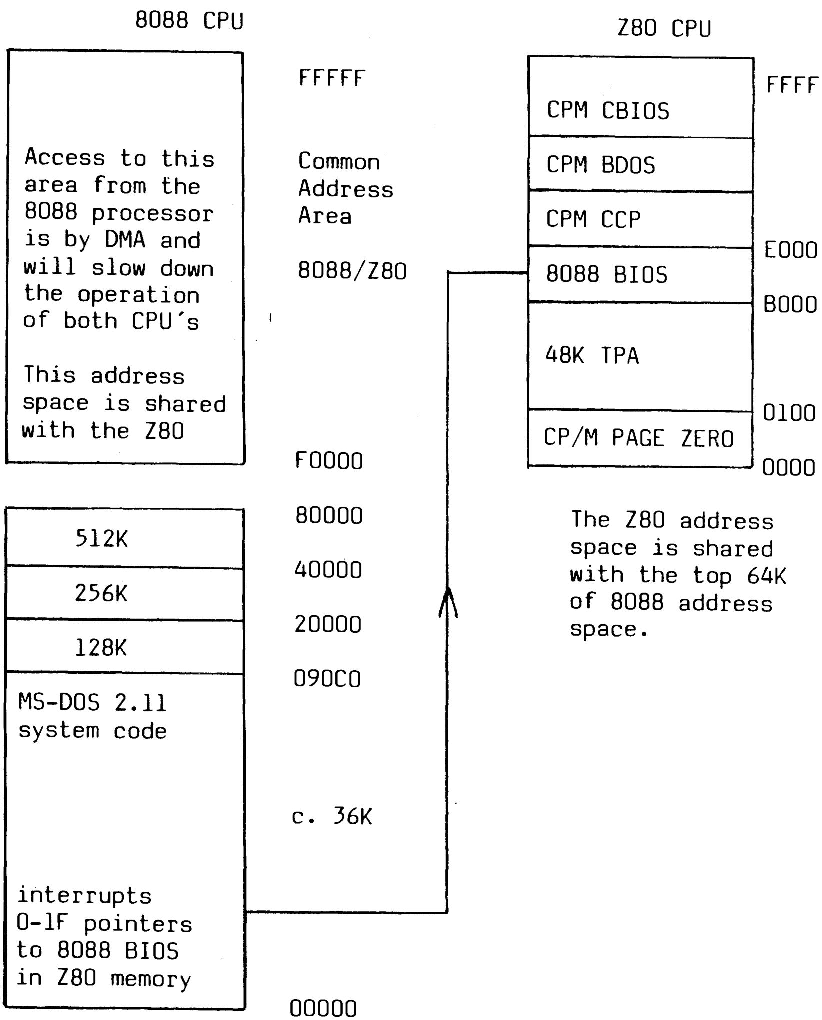
MS-DOS version 2.11 has been implemented on the P2000C.

The MS-DOS implementation on the P2000C is made so that input/output is performed via the 8088 BIOS (part of which resides in the 8088 memory, and part of which is in the Z80 memory). The 8088 BIOS will then communicate via the normal P2000C ROM BIOS to the attached peripherals.

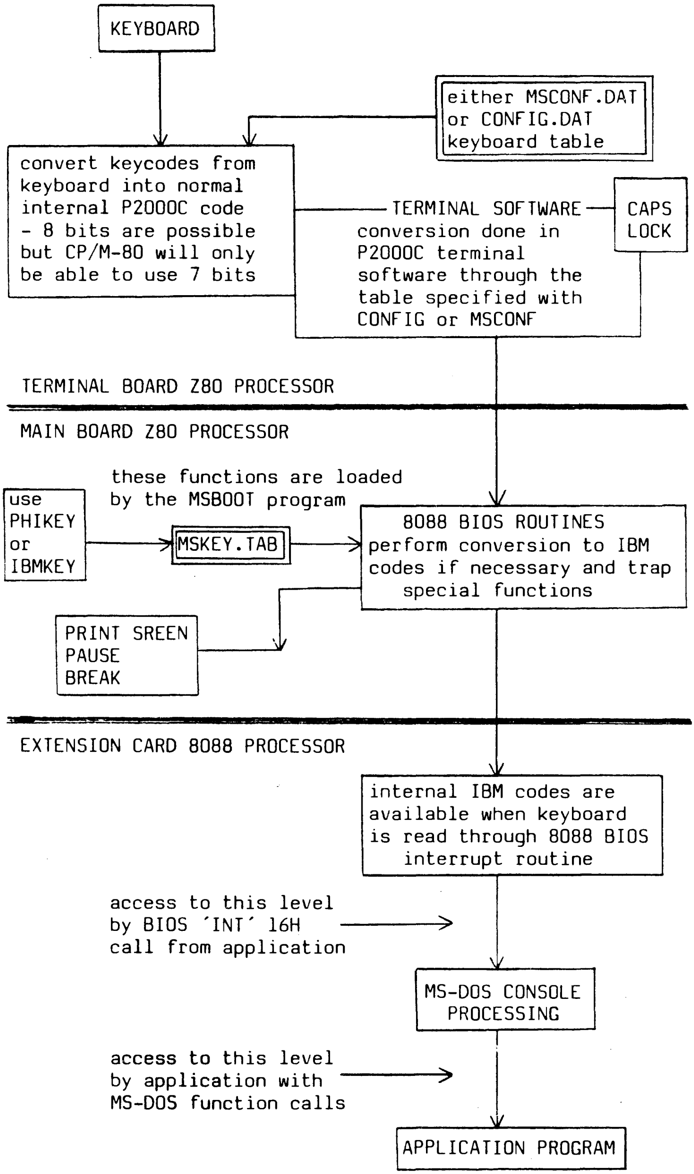
4.2 System Structure



4.3 Memory Map



4.4 Data Flow and Data Mapping



5 DISK ACCESS

The following formats are supported:

P2000C 800k floppy

80 track/sector, double sided, 10 sectors/track,
512 bytes/sector

Bios parameter block (BPB):

Bytes per sector:	0200H (word)
Sectors per allocation unit:	02H (byte)
Reserved sectors:	0001H (word)
Number of FATs:	02H (byte)
Number of root dir. entries:	00C0H (word)
Number of sectors in log. image:	0640H (word)
Media descriptor:	FBH (byte)
Number of sectors per FAT:	0004H (word)
Sectors per track:	000AH (word)
Number of heads	0002H (word)
Number of hidden sectors:	0000H (word)

Disk structure:

Logical address	Contents
0000-01FF	Booting track and BPB.
0200-09FF	First file allocation table (FAT)
0A00-11FF	Second file allocation table (FAT)
1200-29FF	Directory
2A00-C7FFF	Data

P2000C hard disk format with variable length

Bios parameter block (BPB):

```
  Bytes per sector:                      0200H (word)
  Sectors per allocation unit:             ??H (byte)
        (to be patched by MSUTIL)
  Reserved sectors:                      0001H (word)
  Number of FATs:                          02H (byte)
  Number of root dir. entries:           00C0H (word)
  Number of sectors in log. image:       ????H (word)
        (to be patched by MSUTIL)
  Media descriptor:                        FAH (byte)
  Number of sectors per FAT:             ????H (word)
        (to be patched by MSUTIL)
  Sectors per track (virtual):           0010H (word)
  Number of heads (virtual):             0100H (word)
  Number of hidden sectors:              0000H (word)
```

Disk Access

IBM 160k floppy

40 track/sector, single sided, 8 sectors/track, 512 bytes/sector

Bios parameter block (BPB):

Bytes per sector:	0200H (word)
Sectors per allocation unit:	01H (byte)
Reserved sectors:	0001H (word)
Number of FATs:	02H (byte)
Number of root dir. entries:	0040H (word)
Number of sectors in log. image:	0140H (word)
Media descriptor:	FEH (byte)
Number of sectors per FAT:	0001H (word)
Sectors per track:	0008H (word)
Number of heads	0001H (word)
Number of hidden sectors:	0000H (word)

4

Disk Access

IBM 180k floppy

40 track/sector, single sided, 9 sectors/track
512 bytes/sector

Bios parameter block (BPB):

Bytes per sector:	0200H	(word)
Sectors per allocation unit:	01H	(byte)
Reserved sectors:	0001H	(word)
Number of FATs:	02H	(byte)
Number of root dir. entries:	0040H	(word)
Number of sectors in log. image:	0168H	(word)
Media descriptor:	FCH	(byte)
Number of sectors per FAT:	0002H	(word)
Sectors per track:	0009H	(word)
Number of heads	0001H	(word)
Number of hidden sectors:	0000H	(word)

Disk Access

IBM 320k floppy

40 track/sector, single sided, 8 sectors/track
512 bytes/sector

Bios parameter block (BPB):

Bytes per sector:	0200H (word)
Sectors per allocation unit:	02H (byte)
Reserved sectors:	0001H (word)
Number of FATs:	02H (byte)
Number of root dir. entries:	0070H (word)
Number of sectors in log. image:	0280H (word)
Media descriptor:	FFH (byte)
Number of sectors per FAT:	0002H (word)
Sectors per track:	0008H (word)
Number of heads	0002H (word)
Number of hidden sectors:	0000H (word)

Disk Access

IBM 360k floppy

40 track/sector, single sided, 9 sectors/track
512 bytes/sector

Bios parameter block (BPB):

Bytes per sector:	0200H (word)
Sectors per allocation unit:	02H (byte)
Reserved sectors:	0001H (word)
Number of FATs:	02H (byte)
Number of root dir. entries:	0070H (word)
Number of sectors in log. image:	02D0H (word)
Media descriptor:	FDH (byte)
Number of sectors per FAT:	0002H (word)
Sectors per track:	0009H (word)
Number of heads	0002H (word)
Number of hidden sectors:	0000H (word)

6 MS-DOS BIOS (IBM BIOS EMULATION)

The interrupt vectors at the bottom of the 8088 memory will transfer control via a small portion of code in the 8088 to the BIOS which has been loaded in the Z80 memory space and which emulates where possible the IBM ROM BIOS.

The following software interrupts are supported:

- Print screen (INT 05H)
- Execute Z80 subroutine (INT 07H) P2000C onl
- Invalid operation (INT 0EH) P2000C onl
- Video output (INT 10H)
- Equipment check (INT 11H)
- Memory size (INT 12H)
- Disk (INT 13H)
- Communications (INT 14H)
- Keyboard (INT 16H)
- Printer (INT 17H)
- Bootstrap (INT 19H)
- Time of day (INT 1AH)

The following hardware interrupts are supported:

- Keyboard break (INT 1BH)
- Timer tick (INT 1CH)

2

MS-DOS BIOS (IBM BIOS Emulation)

6.1 INT 05H - Print screen

The screen contents will be printed. When a printer timeout error occurs, the beeper will sound.

6.2 INT 07H - Execute Z80 subroutine

On entry:
  BX = subroutine address in Z80 range
  AL = input value to the subroutine
       in register 'A'.
On exit:
  AL = value of register 'A' at exit
       from the subroutine.

To define the free space available for Z80 subroutines, a pointer to the first location occupied by the MS-DOS BIOS is set up at F000:6 and 7. The 8088 processor can use the range from F000:100 to the first occupied location-1.

6.3 INT OEH - Trap Illegal Call

This interrupt is trapped by the 8088 BIOS, which will display on the screen the message:

'ILLEGAL INTERRUPT'

This is used to trap programs that wander into memory which has not been loaded by the program. Unused memory that is filled with 'INT OEH' instructions will cause a jump to this entry when illegally called by an application program.

6.4 INT 10H - Video output

AH=0 Set mode

On entry:
  AL=2 for 80*25 Character mode
  AL=5 for 512*252 Graphics mode

AH=2 Set cursor position

On entry:
  DH=row
  DL=column (0,0 is upper left)
                (paging is not supported)

AH=3 Read cursor position

On exit:
  DH = row
  DL = column
  CH and CL are cleared (no cursor mode support)

AH=6 Scroll up screen

On entry:
  AL = number of lines, if AL=0 blank whole windo
  CH = row of upper line of scroll window
  DH = row of lower line of scroll window

Input lines at the bottom of the window are blanked.

The column positions CL and DL are considered only if CH = DH (window in one line). In this case the field CL to DL is cleared.

MS-DOS BIOS (IBM BIOS Emulation)

AH=7 Scroll down screen

On entry:
  AL = number of lines, if AL=0 blank whole windo
  CH = row of upper line of scroll window
  DH = row of lower line of scroll window

Input lines at the top of the window are blanked.

The column positions CL and DL are considered only if CH = DH (window in one line). In this case the field CL to DL is cleared.

AH=8 Read attribute/character
      at current cursor position

On exit:
  AL = character read
  AH = attribute of character read
       bit-7 is set for blink
       bit-0 is set for underline
       bit-3 is set for high intensity
       bits 6, 5 and 4 are set for invert

AH=9 Write attribute/character
      at current cursor position

On entry:
  AL = character to write
  CX = count of characters to write
  BL = attribute of character
       bit-7 is set for blink
       bits 6-5-4 have the value 111 for invert
       bit-3 is set for high intensity
       bits 2-1-0 have the value 001 for underlin

AH=12 (0CH) Write dot

On entry:
  DX = row number
  CX = column number
  AL = 0 for clear dot, non-zero for set dot

AH=13 (0DH) Read dot

On entry:
  DX = row number
  CX = column number

On exit:
  AL = 0 if dot cleared, non-zero if set

AH=14 (0EH) Write teletype

On entry:
  AL=character to type

All the characters are sent to the terminal board, allowing all the terminal control sequences of the P2000C to be used.

Note : After a terminal reset (18H) the terminal will be in the 24 line mode. To go back to the 25 line mode issue ESC (01BH) followed by 19H.

AH=15 (0FH) Read current video state

On exit:

```
  for 80*25 char mode:
  AL = 2 similar to the mode byte used on entry
         to 'set mode' (AH=0)
  AH = 80 (50H) as screen width

  for graphics (512*252) mode:
  AL = 5 similar to the mode byte used on entry
         to 'set mode' (AH=0)
  AH = 64 (40H) as screen width
```

6.5 INT 11H - Equipment check

```
On exit:
  AX = attached IO
       as indicated by the following bits

  bit-15      = 0
  bit-14      = 1 (one printer)
  bit-13..10  = 0 not used
  bit-09      = 1 (one V24)
  bit-08      = 0
  bit-07-06   = number of floppies - 1
  bit-05-04   = 1,1 (80*25 black & white video)
  bit-03-02   = 1,1
  bit-01      = 0
  bit-00      = 1 if IPL from floppy
```

MS-DOS BIOS (IBM BIOS Emulation)

6.6 INT 12H - Memory size

On exit:
AX = available RAM size in kbytes.

6.7 INT 13H - Diskette input/output

AH=0 Reset diskette system

This command has no effect (the disk will be reset internally as required by the BIOS).

6.7.1 Floppy Disk Access

A special byte at FF00:FFCB (in the common memory) is used to tell the Z80 CPU the track density of the disk in question: the 8088 (system) program must set it to 0 for 96 tpi (P2000C disk) and 1 for 48 tpi (IBM disk).

AH=1 Read the status of the disk system

On exit:

```
  AL = status byte  80H = timeout
                    40H = seek error
                    20H = NEC controller error
                    10H = CRC error
                    04H = sector not found
                    03H = write protect error
                    02H = address mark not found
                    01H = bad command received
```

AH=2 Read the desired sectors into memory

On entry:
  DL = Drive number (0-3 allowed, not checked)
  DH = Head number (0-1 allowed, not checked)
  CH = Track number (0-79 allowed, not checked)
  CL = Starting sector number (1-0A allowed, not checke
  AL = Number of sectors to read (max 0AH, not checked)
       (ES:BX) address of buffer

On exit:
  AH = Status of operation (See Chapter 6.7.1, AH=1)
  CY = 0 if no error (AH=0)
  CY = 1 if error found
  AL = number of sectors read (valid if no error)

AH=3 Write the desired sectors from memory
     (For parameters see Chapter 6.7.1, AH=2)

AH=4 Verify the desired sectors
     (For parameters see Chapter 6.7.1, AH=2)
     The specified sectors are checked for disk error.

AH=5 Format the desired track

The parameters are listed above in the section 'AH=2'. Here, only the drive, head and track information is used. The selected track is formatted with 10 sectors/track, 512 bytes/ sector and 0F6H as the format pattern.

Note: To use a disk formatted by this command it is necessary to write the empty file allocation tables and directory onto the disk. To create a system disk it is necessary to write the bootstrap sector onto the disk, and to use the MS-DOS SYS command to copy the required system files onto the new disk.

6.7.2 Hard Disk Access

Because of the different hard disk structure (basically a CP/M device with 256 bytes/sector format) it is NOT intended to have an IBM-like calling structure. The aim is to simplify the interrupt interface to the MS-DOS I/O.

Because the MS-DOS range on the hard disk is located somewhere in the CP/M range an appropriate offset is added to the called value. This offset is determined at booting time.

The operating system has a limit to the number of records that it can handle. This is because of the system's fixed internal representation. To achieve a larger addressable capacity, each record requested by the operating system will cause two records to be read from the hard disk. So, we arrive at a logical sector size of 512 bytes and a physical sector size of 256.

MS-DOS BIOS (IBM BIOS Emulation)

The following commands are implemented:

AH=2 Read the desired sectors into memory

On entry:
DL = Drive number (80H fix for hard disk)
DH = Virtual head number (0-7FH)
CH = Virtual track number (0 - FFH)
CL = Virtual sector number (1 - 1FH)
AL = Number of sectors to read (1 - 1FH)
(ES:BX) address of buffer

On exit:
AH = Status of operation
(0 if no error, 0FFH if error)
CY = 0 if no error (AH=0)
CY = 1 if error found

AH=3 Write the desired sectors from memory

For parameters, see the read command (Chapter 6.7.2, AH=2)

AH=4 Verify the desired sectors

For parameters, see the read command (Chapter 6.7.2, AH=2)

The specified sectors are checked for disk error.

AH=8 Return drive parameters

It is a dummy call to avoid the 'ILLEGAL CALL's when an IBM PC disk is booted.

On exit:
  AH=1 (Bad command)

MS-DOS BIOS (IBM BIOS Emulation)

6.8 INT 14H - Communication

AH=0 Initialize the communication port

On entry:

```
  AL=Initialization parameter byte
  Bits:  7    6    5    4    3    2    1    0
         -Baud rate-  Parity Stopbit Word-length
         000 -  110   X0-no    0-1   10-7bits
         001 -  150   01-odd   1-2   11-8bits
         010 -  300   11-even
         011 -  600
         100 - 1200
         101 - 2400
         110 - 4800
         111 - 9600
```

This above baud rate overwrites the one defined in the CP/M configuration program.

On exit:

Conditions are set as in the call to command status (see Chapter 6.8, AH=3)

AH=1 Send the character in AL

On exit:

bit-7 of AH is set if the routine was unable to send the character within 20 seconds. The other bits are set as in the status request command. (See Chapter 6.8, AH=3)

AH=2 Receive one character into AL

On exit:

bit-7 of AH is set if the routine was unable to receive the character within 20 seconds. The other bits are set as in the status request command. (See Chapter 6.8, AH=3.)

AH=3 Returns the command port status in (AX)

On exit:

AH register:

```
  Bit-7      = timeout
  Bit-6      = transmitter ready
  Bit-5      = same as bit-6
  Bit-4      = reserved for break detect
  Bit-3      = framing error
  Bit-2      = parity error
  Bit-1      = receiver overrun
  Bit-0      = receiver ready
```

AL register:

```
  Bits 7-6   = 0
  Bit-5      = reserved for data carrier detect (DCD
  Bit-4      = reserved for clear to send (CTS)
  Bits 3-0   = 0
```

6.9 INT 16H - Keyboard

AH=0 Read the next character from the keyboard buffer

A character is read from the terminal board and converted through the special keyboard table read from the MSKEY.TAB file. The table has three bytes output for each character:

ASCII code
Position code
Shift code

The keyboard codes received from the terminal board are converted through this table.

On exit:
AL = ASCII code
AH = position code

There are some special keys:
80H: Print screen
(the screen contents are printed)
9AH: Wait
(waits until another key is pressed)
9CH: Break
(produces a 01BH interrupt on the 8088 side and gives 0 for ASCII and position code
F3H: ALT
(the first 3 numeric keys are considered as decimal values to let the user enter any ASCII value on the keyboard. The position code is 0; the shift code will not change)

AH=1 Read keyboard status

On exit:

```
  Z flag = 1  no character available
  Z flag = 0  character available
  If Z flag = 0, the character remains in the
                 buffer
  AL = ASCII code
  AH = position code
```

AH=2 Read shift status

On exit:

```
  AL  = character shift status as read from the
        translation table 'MSKEY.TAB'
```

6.10 INT 17H - Printer

AH=0 Print character

On entry:
  AL = character

On exit:
  AH = 10H if no error
     = 01H if timeout

AH=2 Read the printer status byte

On exit:
  AH = 10H if ready
     = 90H if busy

6.11 INT 19H - Bootstrap

If the system has been booted from a floppy disk then the message is displayed:

'Insert MS-DOS disk in drive-1,
type return to boot'

The system then continues to reboot as it would during the normal startup procedure.

6.12 INT 1AH - Read/set timer

AH=0 Read the current clock setting

```
On exit:
  CX = high portion of count
  DX = low portion of count
  AL = 0 if timer has not passed 24 hours
       since the last read
     = the number of passed 'midnights'
       since the last read
```

AH = 1 Set the current clock

```
On entry:
  CX = high portion of count
  DX = low portion of count
```

The count frequency is 1193180/65536 (about 18.2) Hz.

Each time the internal timer is incremented, a 'timer-tick interrupt' (INT 1C) is generated on the 8088 side.

7 RUNNING APPLICATION SOFTWARE

The term 'IBM compatibility' has to be explained further. Compatibility means ensuring that a lot of internal features are similar between the 8088 Copower implementation of MS-DOS and the IBM PC range of computers with PC DOS.

This chapter explains how compatibility affects the choice of software to run on the machine. We shall explain the matter so that you can decide whether it would be better to use the IBM-compatible key codes and screen control or the normal P2000C control codes and escape sequences.

Application programs designed specifically for the IBM PC and PC DOS will usually perform screen control via the IBM ROM BIOS (such actions as clear screen and scrolling), and be supplied with only an installation option to allow selection of items such as the display board and monitor. It is assumed that the normal IBM PC keyboard will be available.

To run this kind of software, it is essential to use the IBM PC-like configuration which is possible on the 8088 Copower MS-DOS implementation, so that the software can get the correct characters from the keyboard and display them on the screen. The P2000C character set cannot display all the IBM characters, but it can be configured so that most are similar. As for the others, a representation of the most suitable character has been configured.

Running Application Software

Many application programs which are available in an IBM PC (with PC DOS) version expect 100% compatible hardware to be present, and will also expect direct access to the video memory. This is not possible with the Copower board.

Application programs written in BASIC on the IBM PC will usually perform screen control via the clear screen (CLS) or set cursor position (LOCATE) functions. These functions are not available under normal Microsoft BASIC, and so the programs would have to be converted to perform screen control by control codes and escape sequences. Screen control can be performed in one of two ways on the Copower board - through an installable ANSI terminal driver or via the normal P2000C escape sequences and control codes - but not both at the same time.

IBM PC-compatible machines are usually supplied with an ANSI installable device driver which will accept ANSI escape sequences and convert them into the appropriate screen control codes for the machine. An ANSI device driver (ANSI.SYS) is supplied with the 8088 Copower MS-DOS system.

The IBM PC only recognizes the control codes of Backspace, Carriage Return, Line Feed and Bell. All other characters in the ASCII range 0 to 31, when sent to the screen, would normally be displayed as one of the characters in the character table. On the P2000C, many more characters are recognized as control characters, and would normally be intercepted by the terminal instead of being displayed as a character.

To overcome this problem, the ANSI device driver has been modified to intercept these control characters and perform an internal conversion, so that the 'mosaic character' representation can be used and let the characters be displayed. Therefore, the normal P2000C control characters will not be available when the ANSI driver is installed (unless the application program uses the BIOS call, software interrupt 10H - AH=0E).

Many application programs are available in both a general MS-DOS form and in a specific PC DOS form. If you have a choice, it is normally better to use the general MS-DOS version.

This version will usually come with an install program that allows the selection of screen control codes and escape sequences, so that it can be configured either for the standard ANSI sequences or for the P2000C sequences and control codes. If the package is configured for the latter, the ANSI device driver must NOT be installed. If it is installed for the former, the IBM PC-like keyboard option must be configured on the P2000C.

When a general MS-DOS version of a program is installed to give ANSI sequences (forcing the user to select an IBM PC-like keyboard and screen, so that the ANSI driver can be used), it may not be able to recognize the codes generated by the function keys (cursor movement, for instance) because, being general purpose, the program must allow operation on a normal terminal with only the standard 7-bit ASCII characters. It will usually be better to ccnfigure general MS-DOS packages to run under the normal P2000C key codes and screen codes.

The internal codes used in PC mode are incompatible with those used in the P2000C for characters which are outside the normal US ASCII character set. This means that files containing character data created in PC mode will not appear correct when used in P2000C mode if any of the national language characters, or characters with internal 'IBM PC' code greater than 127, are used - and vice versa. An example of this is German text containing characters with umlauts. On the IBM PC, these would be represented by a code greater than 127, and so would appear false when viewed on a system running with 7-bit ISO codes.

The difference in character sets means that you should use either PC mode or P2000C mode, but you should not mix operations between them.

The character set defined under CP/M-80 for the P2000C is the 7-bit ISO code, allowing the normal US ASCII characters, with national characters if applicable.

It is possible to edit the configuration tables under CP/M-80 to make alterations, adding 8-bit definitions if required.

8 USING Z80 IN THE BACKGROUND

8.1 Normal Processing

It is possible to run both the 8088 and the Z80 at the same time. Z80 code has to be loaded into the TPA (Transient Program Area) from the MS-DOS environment and called via the 8088 BIOS interrupt vector.

There is no protection to avoid a conflict of access from both the 8088 and the Z80 to the peripherals at the same time. It is possible for one part of the code to attempt an access in conflict with the other. It is the responsibility of the programmer to make sure that such a conflict is avoided.

The ability to run background Z80 code while the 8088 is active in the foreground is useful in the implementation of some programs for the 8088, using the remainder of the 30K TPA as a buffer; for example, the running of complex communications protocols. Code like this can run totally in the background, and not need disk access that might conflict with any foreground activity. No access to the screen or keyboard (peripherals used by the 8088) would be necessary.

The 8088 BIOS code running in the Z80 processor contains code to trap a jump to 38H in the Z80 memory. When this occurs the message 'Program hangs at 38H. Please Reset' appears on the screen. This feature will often catch a Z80

Using Z80 in the Background

program if it fails, because the program tends to execute an accidental jump to 38H (OFFH) eventually.

The Z80 program code can be loaded into the CP/M-80 TPA from an MS-DOS program, and executed from the MS-DOS evironment on request by the interrupt 'INT 07' instruction. The 8088 registers will define the Z80 address at which execution is to begin, and will allow a byte as an input parameter to be passed to the Z80 code. It will probably be necessary to transfer the code from a CP/M-80 diskette onto an MS-DOS diskette, using the MSUTIL program, so that the program code can be read as data and transferred via the MS-DOS system into the CP/M-80 TPA.

While the MS-DOS operating system is running in the foreground, the 8088 BIOS must be active and in control of the Z80 processor, so that I/O requests from MS-DOS can be processed.

While the Z80 processor is running, the 8088 processor will be halted. To avoid the possibility of the MS-DOS system 'freezing', the Z80 subroutines should return control after a reasonable time. It is possible to operate more flexibly if extensive use is made of interrupt handlers in the Z80 code, so that the code is 'driven' from the hardware rather than from the MS-DOS side.

8.2 Communications Support

For simple asynchronous communications, the interrupt 14H has been implemented to provide the status, character receive and character send functions. The interface to this interrupt call is the same as that on the IBM PC. However, this may not be adequate to support many of the communications packages available for the IBM PC. But it is possible to implement more advanced features, as we shall describe below.

An MS-DOS applications program cannot directly access the communications ports through the 8088 I/O space. It is possible, however, for the application program to call a routine which exists in the Z80 memory and have that routine executed by the Z80 processor.

This Z80 routine can access the communications ports directly, so it is possible to implement communication drivers in Z80 code. The communication with and transfer of control to the Z80 routine is via an interrupt vector through the 8088 BIOS.

The overhead of the call to the Z80 routine may mean that for high baud rates (where no protocol for transfer control, either software or electrical, is implemented), you must set up an interrupt-driven received character queue in the Z80 memory space, with start and end queue pointers at known locations. The MS-DOS application can then read the characters efficiently, directly from the 8088 memory space which will overlap the Z80 space.

4

Using Z80 in the Background

The Z80 routine can be placed in memory from the MS-DOS application program. The Z80 code can be written to the memory starting at address F000:09H up to but not including the position pointed to by the word in F000:06H-F000:07H. To write above this limit is not allowed because it would overwrite the 8088 BIOS routines.

1

Utility Programs

9 GENERAL

The utility programs associated with the 8088 Copower Board can be divided into two sections:

- those running under CP/M and
- those running under MS-DOS

The names of these utilities and their related files are given in the following table:

CP/M Utilities	Related Files	
MSUTIL	MSUTIL.COM	
	MSUTIL.MSG	
	MSUTILT.MAC	
	MSUTILM.MAC	
TEST88	TEST88.COM	
MSCONF	MSCONF.COM	
	MSCONF.DAT	
	IBMKEY.TAB	) See Note below:
	PHIKEY.TAB	)
	CONFIG.MSG	
MSBOOT	MSBOOT.COM	

MS-DOS Utilities	Related Files
MSSORT/PCSORT	MSSORT.EXE
	PCSORT.EXE
CONFIG.SYS	CONFIG.SYS
ANSI.SYS	ANSI.SYS
CLEAR SCREEN	CLR.COM
REBOOT	REBOOT.COM
DEBUGGER	DEBUG.COM

Note: MSKEY.TAB = IBMKEY.TAB OR PHIKEY.TAB depending on keyboard requirements

3

Utility Programs

9.1 CP/M Utilities

9.1.1 MSUTIL

The utility MSUTIL is an extension of the CP/M utility UTIL and is used to format and copy MS-DOS and CP/M floppy disks, and to transfer files between the two systems. It is also used to configure a hard disk for MS-DOS operation.

The program is driven by menus and prompts. A complete description of the formatting and copying procedures is not required. Points of interest are given in the following paragraphs.

The program is called up, under CP/M, with:

MSUTIL [CR]

and the following menu displayed:

```
SELECT:

        1 - FORMAT FLOPPY DISK

        2 - COPY FLOPPY DISK

        3 - CONFIGURE HARD DISK FOR MS-DOS

  to escape enter <ESC> OR <0>
```

Utility Programs

FORMATTING WITH MSUTIL

All connected drives can be used for formatting. The program will access the drives in the order in which they are entered and continue until an empty drive is found. This will be indicated by the message 'DISK ERROR'.

Successful formatting will be indicated by the message 'READY:n'; where n = the drive number. The formatted disk can then be removed and a new disk inserted for formatting on the next cycle.

The program will format the following:

P2000C DOUBLE SIDED	- CP/M or MS-DOS
P2000C 160K	- CP/M
P2000M	- CP/M
P2500 300K	- CP/M
P2500 600K	- CP/M
P3500	- CP/M
P5020 320K	- CP/M
IBM-PC	- CP/M-86 or 360K MS-DOS
RAINBOW 400K	- CP/M
KAYPRO 200K	- CP/M

It must be pointed out that, in the cases listed below, diskettes formatted with this utility may not be fully compatible with their host systems due to the difference in head widths:

P2000C 160K	- CP/M
P2000M	- CP/M
IBM-PC	- CP/M-86 or 360K MS-DOS

Utility Programs

After the whole disk has been formatted, the operating system is checked. If it is MS-DOS, the BPB (BIOS Parameter Block) is written into the first sector. Then two reset FATs (File Allocation Table) and an empty directory are written onto the disk.

With CP/M, no action is taken. It may lead to difficulties if you format IBM PC CP/M-86 disks because they use an extra media identifier byte on the last address of the boot sector. Its value should be 01. To make this correction, load MS-DOS, start DEBUG, put the CP/M-86 disk into drive B and type:

```
L1000,1,0,1 and press carriage return
E 11FF <CR>
E5 01 <CR>
W1000,1,0,1 <CR>
```

COPYING WITH MSUTIL

Selection of SOURCE and DESTINATION disks is made from the following list:

```
P2000C DOUBLE SIDED - CP/M or MS-DOS
P2000C 160K         - CP/M
P2000M              - CP/M
P2500   300K        - CP/M
P2500   600K        - CP/M
P3500               - CP/M
P5020   320K        - CP/M
IBM-PC              - CP/M-86 or 360K MS-DOS
RAINBOW 400K        - CP/M
KAYPRO  200K        - CP/M
```

Utility Programs

The destination disk must be formatted correctly.

If the source and destination disks are of the same type, it is possible to make a physical copy of the source disk by entering 'y' to the prompt:

"Make PHYSICAL (track to track) copy (y/n)?"

This will produce a complete copy, which will include unwanted 'deleted' files.

Entering 'n' to the above prompt - or if the source and destination disks are of different types - will produce a list of the files on the source disk and the possibility to copy all of the files (enter '*'), or a selected file (enter the number to the left of the file name). Leading zeros must be entered.

This method will only copy the actual files and the appropriate directory entries.

If your source disk is MS-DOS and has a subdirectory, you can select this subdirectory by its number. Then it will be displayed and can be selected for copying.

CONFIGURATION OF THE HARD DISK WITH MSUTIL

The P2000C MS-DOS implementation assumes that the hard disk has two partitions, one each for CP/M and MS-DOS.

Utility Programs

CP/M is the primary partition and it is activated first in all cases. The MS-DOS partition is located in the upper part of the CP/M range and above it. The address of the MS-DOS partition is found in the CP/M directory under the filename MSDOS.ALL under USER-31 (normally not accessible).

To prevent CP/M writing into the MS-DOS area the directory entry of the MSDOS.ALL file must declare all MS-DOS blocks as occupied.

After entering the hard disk configuration program, the current CP/M implementation is examined to see if it contains one, and only one, configured hard disk volume in the lower range (5 MB low or 2 MB). If not, the message:

'No harddisk in this CP/M configuration' or
'Too many harddisk volumes in this CP/M configuration'

is displayed and the procedure aborts.

If the CP/M configuration contains 5MB high and/or 8MB hard disk volumes, the user is first warned that they will be overwritten. The procedure will only be continued if the user acknowledges the requirement by typing the word:

'YES'.

Utility Programs

The user is asked to switch the hard disk on and press any key. The hard disk directory is then checked and if it already has the MSDOS.ALL file the user is warned that the whole MS-DOS part of his hard disk will be overwritten. In this case the user has to type 'YES' to continue the procedure.

The program calculates and displays following values:

- maximum hard disk size (HS)
- CP/M volume size (CS)
- highest allocated CP/M group (HC)

The maximum hard disk space (HS - HC) and the minimum hard disk space (HS - CS) are displayed. The user has to enter the required MS-DOS size (between the minimum and maximum values - see example below) and the program then:

- Puts the MSDOS.ALL file into the CP/M directory
- Configures the appropriate MS-DOS Driver Parameter Block and writes it into the booter sector.
- Writes the booter sector into the first MS-DOS location.
- Writes the first and second File Allocation Tables (FAT).
- Writes an MS-DOS directory with two 'deleted' entries and a 'name' entry.
  The 'deleted' entries are prepared for the IO.SYS and MSDOS.SYS entries and the 'name' entry is 'HARDDISK'.

9

If any of these steps fail, the reason is displayed together with the message:

'Program aborted, MSDOS not installed'.

Example:

The user has a 10 MB hard disk and the lower CP/M range is 5 MB long. He has already used 2 MBytes in CP/M and wishes to reserve an additional 1 MByte space for CP/M. Thus he will have to select 7 MB as the MS-DOS size.

If all these steps are successful, the user is informed of the actions to take in order to use the hard disk as the MS-DOS booting device. All the details are in Chapter 2.9.

9.1.2 TEST88

The 8088 maintenance program checks the general hardware functions. These include:

- memory behaviour
- all interrupts (in both directions)
- refresh circuit
- common memory access lock
- the 8087 mathematics coprocessor

It is also possible to start a simple 8088 memory debugger from the maintenance program.

Test Philosophy

The maintenance program is started from the CP/M environment by entering:

TEST88 [CR]

The program first carries out some checks to confirm that the 8088 Copower Board is present and operating. In the case of 'fatal' error, or if the board is not fitted, the following message is displayed:

'NO ANSWER FROM SLAVE BOARD'

Utility Programs

This message could indicate that the board has not been fitted correctly. No further testing is possible!

If the initial checks are completed satisfactorily, the following message is displayed:

'IS THE 8087 MATH COPROCESSOR INSTALLED ? [Y or N]'

If Y(es) additional tests will be carried out, during the test procedure, to check the 8087.

On start-up, the user can make the following selections:

LONG MEMORY TEST	- L
SHORT MEMORY TEST	- S
INT. & COPROC. TEST	- I
REFRESH & LOCK TEST	- R
8088 MEMORY DEBUGGER	- D
EXIT PROGRAM	- 0 or ESC

Testing

SHORT TEST:

The short test is described in full.

The first function of the SHORT TEST is a check of the memory. After calculating the checksum of the Z80 memory the 8088 memory is filled with a test pattern.

The test program runs in the common (Z80) memory

The display line shows the letter 'Z', indicating that the program is running in Z80 memory, followed by a dot '.' as each 16K block is filled.

Z......

The memory contents are then checked.
As each block is checked, one dot is cleared from the right. If an error is found, the character 'e' will be displayed momentarily and the standard error message will be displayed at the end of the test.

The Z80 checksum is then verified. Any error is indicated with the message:

'checksum error:'

Utility Programs

If no fault is found in the 8088 memory, the program is moved to the lower addresses in the 8088 memory and the test is repeated on locations above the program. The display line shows the letter 'E', indicating that the program is running in External (8088) memory, followed by a dot '.' as each block is filled.

ZE......

As each block is checked, one dot is cleared from the right. If an error is found, the character 'e' will be displayed momentarily and the standard error message will be displayed at the end of the test.

The Z80 memory checksum is then verified again; any error is indicated by the display of the message 'checksum error'.

Note: Because of intermediate calculations, a memory error in 8088 memory will also cause a Z80 checksum error to be shown. If both errors are indicated on the same pass the checksum error can be ignored.

Following the memory test the refresh circuitry is checked. This is indicated in the display line as follows:

ZER

All interrupts are then generated in both directions. The display shows:

ZERI

False interrupt numbers are displayed with the common message in the form:

'GGG1XX00'

This display is explained in the functional description.

On completion, a further pass is started with the memory test.

OTHER TEST ROUTINES:

The LONG test is similar to that described above except that all bit-combinations on all the memory addresses are checked.

The individual INT. & COPROC. TEST (interrupt and coprocessor), and REFRESH & LOCK TEST, check those functions only.

All tests can be terminated by entering 'ESC' or '0'.

Utility Programs

Functional Description

REFRESH & LOCK TEST:

The program runs in common memory. The first 16K block of the 8088 memory is filled with a pseudo-random pattern and the program locks the 8088 memory. At this stage the 8088 awaits a fetch from common memory and the 8088 memory is not disturbed (no read or write). The refresh circuit is active to preserve memory contents.

After a few seconds the 8088 is released and the memory contents are verified. Any errors indicate a fault in the refresh circuitry or the dynamic memory.

INTERRUPT & COPROCESSOR TEST:

The whole 8088 memory is first filled with 'E5' with the instruction 'IN A, [E5]' to all unused memory locations.

In the event of a program crash the logic analyser can be triggered on an input instruction.

If the 8087 Math Coprocessor is present a very simple test is made at this stage.

The program then generates an interrupt table in the range 0 - 3FFH with a starting address for each 256 interrupt entries. Interrupt service routines are then generated in the range 400H - BFFH, pointed to by the interrupt table.

Each routine loads the register AL with its own interrupt number and jumps to a common routine which:
- sets a memory byte with the contents of AL
- sets a ready flag
- returns

The Z80 CPU generates an interrupt with a special number, waits for the ready flag and then checks the received number.

If the flag is not set within a set time, or if the received number is incorrect, the interrupt number is noted. An error summary is displayed at the end of the test; for example:

```
PASS  00215  INTERRUPT ERRORS ON :  0010XXXX
```

This would indicate that interrupts are false on the addresses:

```
00100000 - 00101111
```

Utility Programs

MEMORY TESTS:

The memory tests load the memory with a pseudo-random pattern. This is necessary as the sequence must be reproducible for verification.

The base pattern is 96H, inverted for each pass. This gives a base pattern of 96H for odd number passes and 69H for even number passes, allowing each bit to be checked in both directions.
(96H = 10010110: 69H = 01101001).

For the short test, the pattern is XORed with FF and this value is rotated one position left. The result is written to the next memory cell. One permutation is excluded to give a 7 byte cycle:

D2, A5, 4B, 2D, 5A, B4, 69

For the following short test pass, the values (inverted) will be:

2D, 5A, B4, D2, A5, 4B, 96

The long test works in a similar way, except that the XORed value is counted down from FF to 0 after every test cycle. For the first test the base pattern is the same as for the short test (96 XOR FF = 69). For the second cycle 96 XOR FE = 68. The permutation cycle is:

D0, A1, 43, 0D, 1A, 34, 68

For the following long test pass the values are again inverted.

The short test checks only one possibility for each memory cell while the long test checks every possible combination. For this reason the long test requires about 255 times longer than the short test.

The checksum verification tests for any unwanted interaction between the memories or the processors. A checksum error means that the 8088 has written into the common memory, indicating a memory separator circuit defect or processor interference.

In the event of an error the 8088 sends the character 'e', and a summary of any false bit patterns and addresses are displayed in the form:

```
PASS  00123:  ERROR AT ADDRESS:
              0000 11110000  XXXXXXXX:  G10G10XX
               (0)  (F) (0)  (00 - FF)
```

This indicates errors in the range 0F000 - 0F0FF. The bit summary:

```
bit 7 6 5 4 3 2 1 0
    - - - - - - - -
    G 1 0 G 1 0 X X
```

indicates that:
- bits 7 & 4 are always OK (G)
- bits 6 & 3 are stuck at 1 (1)
- bits 5 & 2 are stuck at 0 (0)
- bits 1 & 0 are faulty in both directions (X)

If no errors are found the following pass starts automatically. The display line is cleared but any error messages are scrolled and accumulated on the screen. If a printer is connected, all messages are printed.

The program is not crash-proof as it runs in RAM. If it does not respond to 'ESC' or '0' (to terminate test), a total failure is indicated. The program cannot run in common memory.

Utility Programs

8088 MEMORY DEBUGGER:

On entering the debugger, the following prompt message is displayed:

```
'Commands: C,D,F,G,M,S,SK,O,"esc", ?
 Entry: Command Start_address End_address Other_info'
```

The command code may be followed by a space (not necessary). The separator between the operands may be a space or a comma. An automatic caps-lock function is implemented, i.e., lower case characters cannot be entered.

The command line format differs from that in the P2000 Maintenance Program Debugger.

The commands, with examples, are shown below. 'CR' indicates the CARRIAGE RETURN key.

COMMAND C: - COMPARE MEMORY BLOCKS

Example: C 14000 14FFF 5000

This command will compare the block 14000 - 14FFF with the block 5000 - 5FFF. Any differences will be displayed.

COMMAND D: - DISPLAY MEMORY BLOCKS

Example: D 6000 607F

The content of locations 6000 to 607F is displayed (HEX and ASCII). The program stores the last displayed address and the length of the block. Therefore, to see the next block (6080 to 60FF), the user has only to type D and 'CR'. D and 'CR' as an initial entry displays the contents of locations 0000 to 00FF. (0000 is the first address of the default disk buffer.)

A long display may be stopped and restarted with CTRL-S. Any other character cancels the command.

COMMAND F: - FILL MEMORY BLOCK WITH ONE BYTE

Example: F 8233 875D 55

The memory locations 8233 to 875D will be filled with 55H.

COMMAND G: - GO TO AN ADDRESS (CALL A PROGRAM)

Example 1: G 5000

Starts 8088 program at location 5000 and returns immediately to the debugger.

Example 2:

Set up the following program in the 8088 memory from address 00000.

```
  0000 2E                  CS:
  0001 C6 06 0010 12       MOV  BYTE PTR [10],12H
  0006 F4                  HLT
```

In machine code this is:

```
           2E, C6, 06, 10, 00, 12, F4
```

The program will simply set the contents of address 10 to 12H.

See paragraph SET MEMORY !!

```
 SO 'CR'             SET memory from address 00000
 00000 XX 2E 'CR' CS:
 00001 XX C6 'CR'
 00002 XX 6  'CR'
 00003 XX 10 'CR'
 00004 XX 0  'CR'
 00005 XX 12 'CR'
 00006 XX F4 'CR'
 00007 XX .  'CR' quit SET command
```

Utility Programs

```
S10 'CR'              SET memory from address 10
00010 XX FF 'CR'      set content of address 10 to FF
00011 XX .  'CR'      quit SET command

G 0 'CR'              start [GO TO] program at
                                             address 0
```

You will now return automatically to the 8088 Memory Debugger

```
S10  'CR'             check content of address  10,
                                    it should be 12H
00010 12  ---------   address 10 is set (to 12H)
                                    by the program!!
```

COMMAND M: - MOVE MEMORY BLOCK

Example: M 4000 4FFF 5000

The block 4000 - 4FFF is copied to 5000 - 5FFF.

COMMAND S: - SET MEMORY

Example: S 4000

The entered start_address and its contents will be displayed.

```
4000  21           - original contents 21H
```

The contents can be altered [enter new value], or left unchanged [enter 'CR']. The next address will then be displayed. To cancel the command, enter '.'

```
4000  21  31       - changes contents to  31H
4001  00  10       - changes contents to  10H
4002  50  CR       - leaves contents  as  50H
4003  32  3A       - changes contents to  3AH
4004  55  .        - cancels command
```

COMMAND SK: - SEEK STRING IN MEMORY

Example: SK CD F9 F6

All addresses where the string 'CD F9 F6' occur are displayed.

COMMAND O or "esc": -
EXIT FROM DEBUGGER INTO MENU TABLE

Note: The "O" or "esc" must be followed by CR to close the command line.

9.1.3 MSCONF

This program works in the same way as the normal P2000C configuration program (CONFIG) used in the CP/M environment, except that it uses the data file MSCONF.DAT instead of CONFIG.DAT.

The program is called up, under CP/M, with:

MSCONF [CR]

This utility should only be used when it is required to configure the P2000C in the PC mode. It is used to set up:

- internal codes
- screen codes
- keyboard codes
- printer tables

Use the program CONFIG when the normal P2000C configuration is required.

9.1.4 MSBOOT

The MS-DOS bootstrap program is entered with:

MSBOOT [CR]

The MSBOOT program is used to install software interface support drivers so that peripherals can be accessed from the 8088 Copower Board.

The program then attempts to load and execute a system disk for the MS-DOS operating system.

The MSBOOT program can be started automatically using the 'autostart' feature of CP/M-80 and set up during the normal CP/M configuration.

When booting from a floppy disk it may be found convenient to boot the CP/M system from drive 2 with the MS-DOS system disk in drive 1. This will allow MSBOOT to proceed without having to remove the CP/M system disk.

Information for transferring the MSBOOT program to hard disk is given in the section on MSUTIL.

9.2 MS-DOS Utilities

9.2.1 MSSORT.EXE AND PCSORT.EXE

The appropriate version of the sort program, either MSSORT or PCSORT will be copied to SORT when the installation batch files are used to install the required mode. Run INSTIBM.BAT or INSTPHI.BAT as appropriate.

MSSORT

The program MSSORT.EXE is the normal MS-DOS sort program (SORT.EXE) and has a collating sequence from 0 to 255. It should always be used when in the MS-DOS P2000C mode.

PCSORT

PCSORT.EXE is a variation of the MS-DOS sort program (SORT.EXE) and has a collating sequence designed to suit the IBM PC character codes (for example, "A umlaut" will sort between A and B). This program should be used when in the MS-DOS PC mode.

9.2.2 CONFIG.SYS

This is the file that MS-DOS reads during startup to determine some settings which can be defined by the user. Some default settings have been supplied on the MS-DOS disk in the CONFIG.SYS, INSTIBM.SYS and INSTPHI.SYS files. As these settings may not be appropriate for your particular needs, the CONFIG.SYS file can be edited using the simple line editor EDLIN. After editing it is necessary to reboot MS-DOS before the new settings take effect.

More details of the CONFI.SYS file and how it is used by MS-DOS can be found in Appendix D of the MS-DOS User Guide.

Note: Do not confuse the above "SYS" files with ANSI.SYS which is not a text file.

9.2.3 ANSI.SYS

The program ANSI.SYS is an installable device driver which is used to convert the ANSI escape sequences for controlling the terminal into the control codes and escape sequences required by the P2000C terminal.

This driver does not allow the reassignment of keys or the attachment of text strings to the function keys, as is possible with the IBM PC ANSI driver.

Note: ANSI.SYS is a code file and should not be confused with the other "SYS" files which are text files.

9.2.4 CLR.COM

The program CLR.COM is provided so that a clear screen function is available in the P2000C mode.

The program generates a HEX 0C character and sends it via a BIOS interrupt call to the Z80 terminal software.

The program will not operate correctly if the ANSI device driver is installed.

9.2.5 REBOOT.COM

The program REBOOT.COM is provided to allow a restart of the MS-DOS system without needing to go back to the CP/M environment, and therefore avoids the use of MSBOOT.

When the system is started from a floppy disk the user is prompted as to whether he wishes to load MS-DOS or return to the CP/M environment.

When the system is started from a hard disk the user is not prompted, and the system reloads the MS-DOS immediately. This is because the system is designed to allow an "AUTOSTART" through CP/M and into MS-DOS with no operator intervention other than pressing the RESET switch on the P2000C computer.

9.2.6 MS-DOS DEBUGGER

OVERVIEW OF DEBUG

The DEBUG Utility (DEBUG) is a debugging program that ~~provides~~ a controlled testing environment for binary and executable object files. Note that EDLIN is used to ~~alter~~ source ~~files; DEBUG~~ is EDLIN's counterpart for binary files. DEBUG eliminates the need to reassemble a program to see if a problem has been fixed by a minor change. It allows you to alter the contents of a file or the contents of a CPU register, and then to immediately reexecute a program to check on the validity of the changes.

All DEBUG commands may be aborted at any time by pressing <Ctrl>/<Break>. <Ctrl>/<Num Lock suspends the display, so that you can read it before the output scrolls away. Entering any key other than <Ctrl>/<Break> or <Ctrl>/<Num Lock> restarts the display.

HOW TO START DEBUG

DEBUG may be started two ways. In the first method, you start DEBUG and then enter all commands in response to the DEBUG prompt (a hyphen). In the second method, you enter all commands on the command line used to start DEBUG.

Summary of Methods to Start DEBUG

Method 1	**DEBUG<CR>**
Method 2	**DEBUG [<pathname> [<arglist>]]<CR>**

Method 1: DEBUG

To start DEBUG using method 1, enter:

DEBUG<CR>

DEBUG responds with the hyphen (-) prompt, signaling that it is ready to accept your commands. Since no filename has been specified, current memory, disk sectors, or disk files can be worked on by using other commands.

Warnings 1. When DEBUG (Version 2.0) is started, it sets up a program header at offset 0 in the program work area. You can overwrite the default header if no <pathname> is given to DEBUG. If you are debugging a .COM or .EXE file, however, do not tamper with the program header below address 5CH, or DEBUG will terminate.

2. Do not restart a program after the Program terminated normally message is displayed. You must reload the program with the N and L commands for it to run properly.

Method 2: Command Line

To start DEBUG using a command line, enter:

DEBUG [<pathname> [<arglist>]]<CR>

For example, if a <pathname> is specified, then the following is a typical command to start DEBUG:

DEBUG FILE.EXE<CR>

DEBUG loads FILE.EXE into memory starting at 100 hexadecimal in the lowest available segment. The BX:CX registers are loaded with the number of bytes placed into memory.

An <arglist> may be specified if <pathname> is present. The <arglist> is a list of filename parameters that are to be passed to the program <pathname>. Thus, when <pathname> is loaded into memory, it is loaded as if it had been started with the command:

<pathname> <arglist>

Here, <pathname> is the file to be debugged, and the <arglist> is the rest of the command line that is used when <pathname> is invoked and loaded into memory.

PARAMETERS

All DEBUG commands accept parameters, except the Quit command. Parameters may be separated by delimiters (spaces or commas), but a delimiter is required only between two consecutive hexadecimal values. Thus, the following commands are equivalent:

```
DCS:100 110
D CS:100 110
D,CS:100,110
```

Table 10-1
DEBUG Command Parameters

PARAMETER — **DEFINITION**

<address> — A two-part designation consisting of either an alphabetic segment register designation or a four-digit segment address, and an offset value. The segment designation or segment address may be omitted, in which case the default segment is used. DS is the default segment for all commands except G, L, T, U, and W, for which the default segment is CS. All numeric values are hexadecimal.

For example:

```
CS:0100
04BA:0100
```

The colon is required between a segment designation (whether numeric or alphabetic) and an offset.

<byte> — A two-digit hexadecimal value to be placed in or read from an address or register.

<drive> — A one-digit hexadecimal value to indicate which drive a file is loaded from or written to. The valid values are 0-3. These values designate the drives as follows: 0=A:, 1=B:, 2=C:, 3=D:.

<list> — A series of <byte> values or of <string>s. <list> must be the last parameter on the command line.

Example:

```
FCS:100 L8 42 45 52 54 41 'ABC'
```

Table 10-1 (Continued)
DEBUG Command Parameters

PARAMETER **DEFINITION**

<range>

An address range specifying a lower and upper address. Two formats can be used:

```
<address> <address>      or

<address> L <value>
```

where <value> is the number of bytes in hex to be processed.

```
Example:  CS:100 110
          CS:100 L 10
```

The following is illegal:

```
     CS:100 CS:110
             ^ Error
```

The limit for <range> is 10000 hex. To specify a <value> of 10000 hex within four digits, enter 0000 (or 0).

<sector> <sector>

1- to 3-digit hexadecimal values used to indicate the starting relative sector on the disk and the number of disk sectors to be written or loaded. Relative sectors are obtained by counting the sectors on the disk surface. The first sector (relative sector 0) is at track 0, sector 1, head 0. Numbering continues for each sector on that track and head, and then continues with the first sector on the next head of the same track. When all the sectors on all heads of that track have been counted, numbering continues with the first sector on head 0 of the next track.

<string>

Any number of characters enclosed in quote marks. Quote marks may be either single (') or double("). If the delimiter quote marks must appear within a <string>, the quote marks must be doubled. For example, the following strings are legal:

```
'This is a "string" is okay.'
'This is a ''string'' is okay.'
```

However, this string is illegal:

```
'This is a 'string' is not.'
```

Table 10-1 (Continued)
DEBUG Command Parameters

PARAMETER	DEFINITION
	Similarly, these strings are legal: "This is a 'string' is okay." "This is a ""string"" is okay." However, this string is illegal: "This is a "string" is not." Note that the double quote marks are not necessary in the following strings: "This is a ''string'' is not necessary." 'This is a ""string"" is not necessary.' The ASCII values of the characters in the string are used as a <list> of byte values.
<value>	A hexadecimal value up to four digits used to specify a port number, numbers to be added or subtracted (Hexarithmetic command), number of bytes, or the number of times a command should repeat its functions.

COMMANDS

Each DEBUG command consists of a single letter followed by parameters. Additionally, the control characters and the special editing functions described in Chapter 6 apply inside DEBUG.

If a syntax error occurs in a DEBUG command, DEBUG reprints the command line and indicates the error with an up-arrow (^) and the word Error."

For example:

```
DCS:100 CS:110
          ^ Error
```

Any combination of upper-case and lower-case letters may be used in commands and parameters.

The DEBUG commands are summarized in Table 10-2 and are described in detail on the following pages.

Table 10-2
DEBUG Commands

DEBUG Command	Function
A[<address>]	Assemble
C<range> <address>	Compare
D[<range>]	Dump
E<address> [<list>]	Enter
F<range> <list>	Fill
G[=<address> [<address>...]]	Go
H<value> <value>	Hex
I<value>	Input
L[<address> [<drive> <sector> <sector>]]	Load
M<range> <address>	Move
N<filename> [<filename>]	Name
O<value> <byte>	Output
Q	Quit
R[<register-name>]	Register
S<range> <list>	Search
T[=<address>] [<value>]	Trace
U[<range>]	Unassemble
W[<address> [<drive> <sector> <sector>]]	Write

A (Assemble) Command

Purpose Assembles 8086/8087/8088 mnemonics directly into memory.

Format A[<address>]

Comments If a syntax error is found, DEBUG responds with

```
^Error
```

and redisplays the current assembly address.

All numeric values are hexadecimal and must be 1-4 characters. Prefix mnemonics must be specified in front of the opcode to which they refer. They may also be entered on a separate line.

The segment override mnemonics are CS:, DS:, ES:, and SS:. The mnemonic for the far return is RETF. String manipulation mnemonics must explicitly state the string size. For example, use MOVSW to move word strings and MOVSB to move byte strings.

The assembler automatically assembles short, near, or far jumps and calls, depending on byte displacement to the destination address. These may be overridden with the NEAR or FAR prefix. For example:

```
0100:0500 JMP   502          ; a 2-byte short jump
0100:0502 JMP   NEAR 505     ; a 3-byte near jump
0100:0505 JMP   FAR  50A     ; a 5-byte far jump
```

The NEAR prefix may be abbreviated to NE, but the FAR prefix cannot be abbreviated.

DEBUG cannot tell whether some operands refer to a word memory location or to a byte memory location. In this case, the data type must be explicitly stated with the prefix "WORD PTR" or "BYTE PTR". Acceptable abbreviations are "WO" and "BY". For example:

```
NEG     BYTE PTR [128]
DEC     WO [SI]
```

Utility Programs

A (Assemble) Command

DEBUG also cannot tell whether an operand refers to a memory location or to an immediate operand. DEBUG uses the common convention that operands enclosed in square brackets refer to memory. For example:

```
MOV     AX,21          ; Load AX with 21H
MOV     AX,[21]        ; Load AX with the
                       ; contents of memory
                       ; location 21H
```

Two popular pseudo-instructions are available with Assemble. The DB opcode assembles byte values directly into memory. The DW opcode assembles word values directly into memory. For example:

```
DB      1,2,3,4,"THIS IS AN EXAMPLE"
DB      'THIS IS A QUOTE: "'
DB      "THIS IS A QUOTE: '"
DW      1000,2000,3000,"BACH"
```

Assemble supports all forms of register indirect commands. For example:

```
ADD     BX,34[BP+2].[SI-1]
POP     [BP+DI]
PUSH    [SI]
```

All opcode synonyms are also supported. For example:

```
LOOPZ   100
LOOPE   100
JA      200
JNBE    200
```

For 8087 opcodes, the WAIT or FWAIT must be explicitly specified. For example:

```
FWAIT FADD ST,ST(3)     ; This line will assemble
                        ; a FWAIT prefix
LD TBYTE PTR [BX]       ; This line will not
```

Utility Programs

C (Compare) Command

Purpose Compares the portion of memory specified by <range> to a portion of the same size beginning at <address>.

Format C<range> <address>

Comments If the two areas of memory are identical, there is no display and DEBUG returns with the (-) prompt. If there are differences, they are displayed in this format:

<address1> <byte1> <byte2> <address2>

If only an offset is entered for <range>, the C command assumes the segment contained in the DS register.

Examples The following commands have the same effect:

C100,1FF 300<CR>

or

C100L100 300<CR>

Each command compares the block of memory from 100 to 1FFH with the block of memory from 300 to 3FFH. The segment in the DS register is used by default.

Utility Programs

D (Dump) Command

Purpose Displays the contents of the specified region of memory.

Format D[<range>]

Comments If a range of addresses is specified, the contents of the range are displayed. If the D command is entered without parameters, 128 bytes are displayed at the first address (DS:100) after the address displayed by the previous Dump command.

The dump is displayed in two portions: a hexadecimal dump (each byte is shown in hexadecimal value) and an ASCII dump (the bytes are shown in ASCII characters). Non-printing characters are denoted by a period (.) in the ASCII portion of the display. Each display line shows 16 bytes with a hyphen between the eighth and ninth bytes. At times, displays are split in this manual to fit them on the page. Each displayed line begins on a 16-byte boundary.

Examples If you enter the command:

DCS:100 10F<CR>

DEBUG displays the sixteen memory locations from CS:100 to CS:10F in the following format.

```
04BA:0100 43 61 6E 6E ... 73 70  Cannot open resp
```

If you enter the following command:

D<CR>

the next 128 bytes are displayed. Each line of the display begins with an address, incremented by 16 from the address on the previous line. Each subsequent D (entered without parameters) displays the 128 bytes immediately following those last displayed.

E (Enter) Command

Purpose Enters byte values into memory at the specified <address>.

Format E<address> [<list>]

Comments If the optional <list> of values is entered, the replacement of byte values occurs automatically. (If an error occurs, no byte values are changed.)

If the <address> is entered without the optional <list>, DEBUG displays the address and its contents, then repeats the address on the next line and waits for your input. At this point, the Enter command waits for you to perform one of the following actions:

1. Replace the byte value with a value you enter. Enter the value after the current value. If the value entered in is not a legal hexadecimal value or if more than two digits are entered, the illegal or extra character is not echoed.

2. Press the <Space Bar> to advance to the next byte. To change the value, enter the new value as described in (1.) above. If you space beyond an 8-byte boundary, DEBUG starts a new display line with the address displayed at the beginning.

3. Enter hyphen (-) to return to the preceding byte. If you decide to change a byte behind the current position, entering the hyphen returns the current position to the previous byte. When the hyphen is entered, a new line is started with the address and its byte value displayed.

4. Press <CR> to terminate the Enter command. The <CR> key may be pressed at any byte position.

Example Assume that the following command is entered:

ECS:100<CR>

Suppose DEBUG displays:

```
04BA:0100  EB._
```

To change this value to 41, enter 41 as shown:

```
04BA:0100  EB.41_
```

E (Enter) Command

To step through the subsequent bytes, press the <Space Bar> three times to see:

```
04BA:0100  EB.41   10.    00.    BC._
```

To change BC to 42, enter 42 as shown:

```
04BA:0100  EB.41   10.    00.    BC.42_
```

Now, realizing that 10 should be 6F, enter the hyphen twice to return to byte 0101 (value 10), then replace 10 with 6F:

```
04BA:0100  EB.41   10.    00.    BC.42-
04BA:0102  00.-_
04BA:0101  10.6F_
```

Pressing <CR> changes any entered values in memory, ends the Enter command, and returns to the DEBUG command level.

F (Fill) Command

Purpose Fills the addresses in the <range> with the values in the <list>.

Format F<range> <list>

Comments If the <range> contains more bytes than the number of values in the <list>, the <list> is used repeatedly until all bytes in the <range> are filled. If the <list> contains more values than the number of bytes in the <range>, the extra values in the <list> are ignored. If any of the memory in the <range> is not valid (bad or nonexistent), an error occurs in all succeeding locations.

Example Assume that the following command is entered:

F04BA:100 L 100 42 45 52 54 41<CR>

DEBUG fills memory locations 04BA:100 through 04BA:1FF with the bytes specified. The five values are repeated until all 100H bytes are filled.

G (Go) Command

Purpose	Executes the program currently in memory.
Format	G[=<address> [<address>...]]
Comments	If only the Go command is entered, the program executes as if the program had run outside DEBUG.

If =<address> is set, execution begins at the address specified. The equal sign (=) is required, so that DEBUG can distinguish the start =<address> from the breakpoint <address>es.

With the other optional addresses set, execution stops at the first <address> encountered, regardless of that address' position in the list of addresses to halt execution or program branching. When program execution reaches a breakpoint, the registers, flags, and decoded instruction are displayed for the last instruction executed. (The result is the same as if you had entered the Register command for the breakpoint address.)

Up to ten breakpoints may be set. Breakpoints may be set only at addresses containing the first byte of an 8086 opcode. If more than ten breakpoints are set, DEBUG returns the BP error message (see the error message listing at the end of this chapter).

The user stack pointer must be valid and have 6 bytes available for this command. The G command uses an IRET instruction to cause a jump to the program under test. The user stack pointer is set, and the user flags, Code Segment register, and Instruction Pointer are pushed on the user stack. (Thus, if the user stack is not valid or is too small, the operating system may crash.) An interrupt code (0CCH) is placed at the specified breakpoint address(es).

When an instruction with the breakpoint code is encountered, all breakpoint addresses are restored to their original instructions. If execution is not halted at one of the breakpoints, the interrupt codes are not replaced with the original instructions.

G (Go) Command

Example Assume that the following command is entered:

GCS:7550<CR>

The program currently in memory begins with the current instruction (current address of CS:IP) and executes up to the address 7550 in the CS segment. DEBUG then displays registers and flags, after which the Go command is terminated.

After a breakpoint has been encountered, if you enter the Go command again, the program executes just as if you had entered the filename at the DOS command level. The only difference is that program execution begins at the instruction after the breakpoint rather than at the usual start address.

H (Hex)
Command

Purpose Performs hexadecimal arithmetic on the two values specified.

Format H<value> <value>

Comments First, DEBUG adds the two parameters, then subtracts the second parameter from the first. The results of the arithmetic are displayed on one line; first the sum, then the difference.

Example Assume that the following command is entered:

H19F 10A<CR>

DEBUG performs the calculations and then displays the result:

```
02A9   0095
```

I (Input)
Command

Purpose Inputs and displays one byte from the port specified by <value>.

Format I<value>

Comments A 16-bit port address is allowed.

Example Assume that you enter the following command:

I2F8<CR>

Assume also that the byte at the port is 42H. DEBUG inputs the byte and displays the value:

```
42
```

Utility Programs

L (Load) Command

Purpose Loads a file into memory.

Format L[<address> [<drive> <sector> <sector>]]

Comments If only the L command is entered (no parameters), DEBUG loads into memory starting at location CS:100 the file whose filespec is in the file control block at CS:5C. A file is placed in the file control block by entering the filespec on the command line when DEBUG is started, or by using the NAME command. The L command sets the BX and CX registers to the number of bytes that have been loaded into memory.

If the L command is entered with only the <address> parameter, the file specified in the file control block is loaded into memory at the specified address.

Files with a file extension of .COM are always loaded into memory at location CS:100, regardless of a specified <address> in the L command.

If the file has a .EXE extension, it is relocated to the load address specified in the header of the .EXE file: the <address> parameter is always ignored for .EXE files. The header itself is stripped off the .EXE file before it is loaded into memory. Thus the size of an .EXE file on disk differs from its size in memory.

If the file named by the Name command or specified when DEBUG is started is a .HEX file, then entering the L command with no parameters causes DEBUG to load the file beginning at the address specified in the .HEX file. If the L command includes the option <address>, DEBUG adds the <address> specified in the L command to the address found in the .HEX file to determine the start address for loading the file.

If L is entered with all the parameters, absolute disk sectors are loaded into memory. The sectors are taken from the <drive> specified (the drive designation is numeric where 0=A:, 1=B:, 2=C:, and 3=D:); DEBUG begins loading with the first <sector> specified, and continues until the number of sectors specified in the second <sector> have been loaded.

The maximum number of sectors that can be loaded by an L command is hex 80.

L (Load) Command

Example Assume that the following commands are entered:

A>DEBUG<CR>
-NFILE.COM

Now, to load FILE.COM, enter:

L<CR>

FILE.COM is loaded into memory at CS:100 and DEBUG returns the (-) prompt.

The command:

LCS:100 1 OF 6D<CR>

would load 109 (6D hex) sectors from drive B beginning with logical sector 15 (OF hex) into memory starting at address CS:0100. When the records have been loaded, DEBUG returns the (-) prompt.

M (Move) Command

Purpose Moves the block of memory specified by <range> to the location beginning at the <address> specified.

Format M<range> <address>

Comments Overlapping moves (moves where part of the block overlaps some of the current addresses) are always performed without loss of data. Addresses that could be overwritten are moved first. The sequence for moves from higher addresses to lower addresses is to move the data beginning at the block's lowest address and then to work towards the highest. The sequence for moves from lower addresses to higher addresses is to move the data beginning at the block's highest address and to work towards the lowest.

Note that if the addresses in the block being moved do not have new data moved to them, the data there before the move remains. The M command copies the data from one area into another, in the sequence described, and writes over the new addresses. This is why the sequence of the move is important.

If only an offset is entered for the starting address of the <range> parameter or for the <address> parameter, the M command uses the segment contained in the DS register.

Example Assume that you enter:

MCS:100 110 CS:500<CR>

DEBUG first moves address CS:110 to address CS:510, then CS:10F to CS:50F, and so on until CS:100 is moved to CS:500. You should enter the Dump command, using the <address> entered for the M command, to review the results of the move.

Utility Programs

N (Name) Command

Purpose Sets filenames.

Format N<filespec> [<filespec>...]

Comments The N command performs two functions. First, N is used to assign a filename for a later Load or Write command. If you start DEBUG without naming any file to be debugged, the N<filespec> command must be entered before a file can be loaded. Second, N is used to assign filename parameters to the file being debugged. In this case, Name accepts a list of parameters that are used by the file being debugged.

These two functions overlap. Consider the following set of DEBUG commands:

```
-NFILE1.EXE<CR>
-L<CR>
-G<CR>
```

Because of the effects of the N command, Name performs the following steps:

1. (N)ame assigns the filename FILE1.EXE to the file to be used in any later Load or Write commands.

2. (N)ame also assigns the filename FILE1.EXE to the first filename parameter used by any program that is later debugged.

3. (L)oad loads FILE1.EXE into memory.

4. (G)o causes FILE1.EXE to be executed with FILE1.EXE as the single filename parameter (that is, FILE1.EXE is executed as if FILE1.EXE had been entered at the command level).

A more useful chain of commands might look like this:

```
-NFILE1.EXE<CR>
-L<CR>
-NFILE2.DAT FILE3.DAT<CR>
-G<CR>
```

N (Name) Command

Here, Name sets FILE1.EXE as the filename for the subsequent Load command. The Load command loads FILE1.EXE into memory, and then the N command is used again, this time to specify the parameters to be used by FILE1.EXE. Finally, when the Go command is executed, FILE1.EXE is executed as if FILE1 FILE2.DAT FILE3.DAT had been entered at the MS-DOS command level. Note that if a Write command were executed at this point, FILE1.EXE (the file being debugged) would be saved with the name FILE2.DAT! To avoid such undesired results, you should always execute a N command before either a Load or a Write.

There are four regions of memory that can be affected by the N command:

CS:5C	FCB for file 1
CS:6C	FCB for file 2
CS:80	Count of characters
CS:81	All characters entered

A File Control Block (FCB) for the first filename parameter given to the N command is set up at CS:5C. If a second filename parameter is entered, then an FCB is set up for it beginning at CS:6C. The number of characters entered in the N command (exclusive of the first character N) is given at location CS:80. The actual stream of characters given by the N command (again, exclusive of the character N) begins at CS:81. Note that this stream of characters may contain parameters and delimiters that would be legal in any command entered at the MS-DOS command level.

Example

A typical use of the N command is:

```
A>DEBUG PROG.COM<CR>
-NPARAM1 PARAM2/C<CR>
-G<CR>
```

In this case, the Go command executes the file in memory as if the following command line had been entered:

```
PROG PARAM1 PARAM2/C<CR>
```

Testing and debugging therefore reflect a normal runtime environment for PROG.COM.

O (Output) Command

Purpose Sends the <byte> specified to the output port specified by <value>.

Format O<value> <byte>

Comments A 16-bit port address is allowed.

Example Enter:

O2F8 4F<CR>

DEBUG outputs the byte value 4F to output port 2F8.

Q (Quit) Command

Purpose Terminates the DEBUG utility.

Format Q

Comments The Q command takes no parameters and exits DEBUG without saving the file currently being operated on. You are returned to the MS-DOS command level.

Example To end the debugging session, enter:

Q<CR>

DEBUG is terminated and control returns to the MS-DOS command level.

R (Register) Command

Purpose Displays the contents of one or more CPU registers.

Format R[<register-name>]

Comments If no <register-name> is entered, the R command dumps th register save area and displays the contents of all registers and flags.

If a register name is entered, the 16-bit value of that register is displayed in hexadecimal, and then a colon appears as a prompt. You then either enter a <value> to change the register, or press <CR> if no change is wanted.

The only valid <register-name>s are:

AX	BP	SS	
BX	SI	CS	
CX	DI	IP	(IP and PC both refer to the
DX	DS	PC	Instruction Pointer).
SP	ES	F	

Any other entry for <register-name> results in a BR Error message.

If F is entered as the <register-name>, DEBUG displays each flag with a two-character alphabetic code. To alter any flag, enter the opposite two-letter code. The flags are either set or cleared.

The flags are listed below with their codes for SET and CLEAR:

FLAG NAME	SET	CLEAR
Overflow	OV	NV
Direction	DN Decrement	UP Increment
Interrupt	EI Enabled	DI Disabled
Sign	NG Negative	PL Plus
Zero	ZR	NZ
Auxiliary Carry	AC	NA
Parity	PE Even	PO Odd
Carry	CY	NC

R (Register) Command

Whenever you enter the command RF, the flags are displayed in the order shown above in a row at the beginning of a line. At the end of the list of flags, DEBUG displays a hyphen (-). You may enter new flag values as alphabetic pairs. The new flag values can be entered in any order. You do not have to leave spaces between the flag entries. To exit the R command, press <CR>. Flags for which new values were not entered remain unchanged.

If more than one value is entered for a flag, DEBUG returns a DF Error message. If you enter a flag code other than those shown above, DEBUG returns a BF Error message. In both cases, the flags up to the error in the list are changed; flags at and after the error are not.

At startup, the segment registers are set to the bottom of free memory, the Instruction Pointer is set to 0100H, all flags are cleared, and the remaining registers are set to zero.

Example Enter:

R<CR>

DEBUG displays all registers, flags, and the decoded instruction for the current location. If the location is CS:11A, then the display will look similar to this:

```
AX=0E00 BX=00FF CX=0007 DX=01FF SP=039D BP=0000
SI=005C DI=0000 DS=04BA ES=04BA SS=04BA CS=O4BA
IP=011A  NV UP DI NG NZ.AC PE NC
04BA:011A  CD21          INT     21
```

If you enter:

RF<CR>

DEBUG might display the following flag settings:

```
NV UP DI NG NZ AC PE NC -
```

R (Register) Command

To change the settings, enter any valid flag designation, in any order, with or without spaces.

For example:

NV UP DI NG NZ AC PE NC - **PLEICY<CR>**

DEBUG responds only with the DEBUG prompt. To see the changes, enter either the R or RF command:

RF<CR>

DEBUG displays the new settings:

NV UP EI PL NZ AC PE CY - _

Press <CR> to leave the flags this way.

S (Search) Command

Purpose Searches the <range> specified for the <list> of bytes specified.

Format S<range> <list>

Comments The <list> may contain one or more bytes, each separated by a space or comma. If the <list> contains more than one byte, only the first address of the byte string is returned. If the <list> contains only one byte, all addresses of the byte in the <range> are displayed.

Example If you enter:

SCS:100 110 41<CR>

DEBUG displays a response similar to this:

04BA:0104
04BA:010D

T (Trace) Command

Purpose Executes one instruction and displays the contents of all registers and flags, and the decoded instruction.

Format T[=<address>] [<value>]

Comments If the optional =<address> is entered, tracing occurs at the =<address> specified. The optional <value> causes DEBUG to execute and trace the number of steps specified by <value>.

The T command uses the hardware trace mode of the 8086 or 8088 microprocessor. Consequently, you may also trace instructions stored in ROM (Read Only Memory).

Example Enter:

T<CR>

DEBUG returns a display of the registers, flags, and decoded instruction for that one instruction. Assume that the current position is 04BA:011A; DEBUG might return the display:

```
AX=0E00 BX=00FF CX=0007 DX=01FF SP=039D BP=0000
SI=005C DI=0000 DS=04BA ES=04BA SS=04BA CS=04BA
IP=011A  NV UP DI NG NZ AC PE NC
04BA:011A  CD21           INT     21
```

If you enter:

T=011A 10<CR>

DEBUG executes sixteen (10 hex) instructions beginning at 011A in the current segment, and then displays all registers and flags for each instruction as it is executed. The display scrolls away until the last instruction is executed. Then the display stops, and you can see the register and flag values for the last few instructions performed. Remember that <Ctrl>/<Num Lock> suspends the display at any point, so that you can study the registers and flags for any instruction.

U (Unassemble)
Command

Purpose Disassembles bytes and displays the source statements that correspond to them, with addresses and byte values.

Format U[<range>]

Comments The display of disassembled code looks like a listing for an assembled file. If you enter the U command without parameters, 20 hexadecimal bytes are disassembled at the first address after that displayed by the previous Unassemble command. If you enter the U command with the <range> parameter, DEBUG disassembles all bytes in the range. If the <range> is given as an <address> only, then 20H bytes are disassembled starting at that <address>.

Example Enter:

U04BA:100 L10<CR>

DEBUG disassembles 16 bytes beginning at address 04BA:0100 in the following format:

```
04BA:0100  206472   AND  [SI+72],AH
04BA:0103  69       DB   69
04BA:0104  7665     JBE  016B
04BA:0106  207370   AND  [BP+DI+70],DH
04BA:0109  65       DB   65
04BA:010A  63       DB   63
04BA:010B  69       DB   69
04BA:010C  66       DB   66
04BA:010D  69       DB   69
04BA:010E  63       DB   63
04BA:010F  61       DB   61
```

If you enter:

U04BA:0100 0108<CR>

The display shows:

```
04BA:0100  206472   AND  [SI+72],AH
04BA:0103  69       DB   69
04BA:0104  7665     JBE  016B
04BA:0106  207370   AND  [BP+DI+70],DH
```

If the bytes in some addresses are altered, the disassembler alters the instruction statements. The U command can be entered for the changed locations, the new instructions viewed, and the disassembled code used to edit the source file.

W (Write) Command

Purpose	Writes the file being debugged to a disk file.
Format	W[<address> [<drive> <sector> <sector>]]

Comments

If you enter the W command with no parameters, the file in memory starting at memory location CS:100 is written to disk using the filespec contained in the file control block at CS:5C. The BX and CX registers must be set to the number of bytes to be written.

If the W command is entered with just the <address> parameter, the file starting at memory location <address> is written to disk using the filespec contained in the file control block at CS:5C. Again, the BX and CX registers must be set to the number of bytes to be written.

Note, if a Go or Trace command has been used, the values placed in the BX and CX registers when the file was loaded might have been changed.

When a file is loaded, edited under DEBUG, and then written back to disk with the same filespec, the edited file is written over the original file.

If the W command is entered with all of the parameters, data is written from memory location <address> to the specified <drive> (the drive designation is numeric, where 0=A:, 1=B:, 2=C:, and 3=D:). DEBUG writes the data to disk beginning at the logical sector number specified by the first <sector> parameter and continues to write until the number of sectors specified in the second <sector> parameter have been written.

WARNING Writing to absolute sectors is EXTREMELY dangerous because the process bypasses the file handler.

W (Write) Command

Examples If you enter:

W<CR>

DEBUG writes the file starting at memory location CS:100 to disk using the filespec contained in the file control block at CS:5C. The number of bytes written to disk are contained in the BX and CX registers. DEBUG then display the DEBUG prompt.

Entering:

WCS:100 1 37 2B<CR>

DEBUG writes the contents of memory beginning at address CS:100 to the disk in drive B:. 2BH sectors of data are written to disk starting at logical sector number 37H.

ERROR MESSAGES

Duringthe DEBUG session,you may receive any of thefollowing errormessages. Each error terminates the DEBUG commandunder which it occurred, but does not terminate DEBUG itself.

Table 10-3
DEBUG Error Codes

ERROR CODE	DEFINITION
BF	Bad flag You attempted to alter a flag, but the characters entered were not one of the acceptable pairs of flag values. See the Register command for the list of acceptable flag entries.
BP	Too many breakpoints You specified more than ten breakpoints as parameters to the G command. Reenter the G command with ten or fewer breakpoints.
BR	Bad register You entered the R command with an invalid register name. See the Register command for the list of valid register names.
DF	Double flag You entered two values for one flag. You may specify a flag value only once per RF command.

10 INSTALLATION

10.1 Terminalboard Software

In order to make use of the 8088 Copower Board, it is necessary to have the correct release of the terminalboard software. The Copower Board is compatible with all releases of the terminalboard software from release 1.3 onwards.

To check your terminalboard software release it is simply necessary to press RESET and, before the IPL message is displayed, press the ESC key.

The software release will be displayed as:

TERMINAL SW-Rel: x.x

If the indicated release is lower than 1.3, it will be necessary to replace the terminalboard ROM.
Fitting instructions are given on the following page. Please contact your dealer if you require further information!

10.2 Fitting the Board

With your 8088 Copower Board you will find the P2000C Fitting Instructions for Optional Boards. If you have a copy of this document which was prepared before the 8088 Copower Board was introduced, the method of fitting it is the same as that for the P 2092 Memory Extension Board.

Please follow these instructions carefully.

10.2.1 FITTING THE TERMINALBOARD ROM

The ROM is fitted to a 28 pin I/C socket on the terminal board.

First, remove the sub-chassis as shown in steps 1 and 2 of the Fitting Instructions and identify the existing ROM. This is item 7409 and is located at the bottom right hand corner of the terminal board.

Note the position of the notch on the body of the existing I/C and then carefully remove it. Fit the new I/C, making sure that the pins are not damaged and that the notch is in the same position as on the original.

When the P2000C has been re-assembled, the I/C can be tested by pressing ESC and RESET. The software release number will be displayed and testing carried out.

10.3 System Upgrades

The 8088 Copower Board can be fitted with:

- 2 banks of 64KBit chips or
- 1 or 2 banks of 256KBit chips

These configurations will give:
128, 256 or 512 KBytes of additional RAM.

If the board is upgraded, i.e., if more or larger RAMs are fitted, ensure that all chips are fitted with the same orientation as those originally fitted.

In addition to the RAM, the 8088 is wired to accept an 8087 Mathematics Coprocessor.
Full instructions for fitting this will be supplied by your dealer when the device is purchased.

This maths coprocessor can only be used for application programs that use the special 8087 instructions. Fitting the device will have no effect on normal programs.

4

Hardware Installation

10.4 Testing After Installation

A complete check of the 8088 Copower Board can be carried out by running the program TEST88.

The operation of this program is described in Chapter 9 of this manual.

10.5 Restrictions

If 512KBytes of RAM are fitted together with the 8087 math processor, the environmental specification is down-graded and the equipment must not be used with an ambient temperature above 25` C.

In all cases ensure that the P2000C is operated with the tilt-bar down, to allow for efficient air cooling, and that the cooling slots on the top of the machine are not obstructed.